

TRABAJO DE FIN DE GRADO

Grado en Ingeniería de Computadores

ESCUELA POLITÉCNICA SUPERIOR

Estudio e Implementación de un Planificador de
Trayectorias en el Simulador CARLA con Modificación
de Escenarios y Descripción del Entorno mediante
Modelos Multimodales

Rubén Sanz Noguerales

Tutor: Augusto Luis Ballardini

2026

UNIVERSIDAD DE ALCALÁ

ESCUELA POLITÉCNICA SUPERIOR

Grado en Ingeniería de Computadores

Trabajo de Fin de Grado

**Estudio e Implementación de un Planificador de Trayectorias en
el Simulador CARLA con Modificación de Escenarios y
Descripción del Entorno mediante Modelos Multimodales**

Autor: Rubén Sanz Nogueras

Tutor: Augusto Luis Ballardini

Tribunal:

Presidente: Carlota Salinas Maldonado

Vocal: Antonio Guerrero Baquero

Suplente: Javier Alonso Ruiz

Fecha de depósito: 9 de julio de 2026

“Es posible volar sin motores, pero no sin conocimiento o habilidad”

Wilbur Wright

Agradecimientos

En primer lugar, me gustaría expresar mi agradecimiento a mi tutor, Augusto, por su orientación, dedicación, paciencia y, sobre todo, las facilidades puestas a mi disposición para seguir adelante pese a las limitaciones computacionales que exige trabajar conjuntamente con CARLA y LLMs.

A mi compañero Sinan, por el apoyo mutuo frente a cada nuevo problema y por hacer que este camino haya sido una experiencia compartida mucho más fácil de transitar.

A mis padres y a mi hermana, por ser el mejor refugio y el mejor ejemplo a seguir. La muestra de que la disciplina y la búsqueda constante de la excelencia son el motor para alcanzar cualquier meta propuesta, y la prueba de que nada de esto importa sin unas inmensas ganas de disfrutar del camino.

Finalmente, quiero dar las gracias a todas las personas que me han inspirado a ver este trabajo no solo como un desafío técnico, sino como una oportunidad para acercar la tecnología a la sociedad. El proyecto Humanav, que persigue una experiencia de conducción autónoma más humana y explicable, es la muestra de que el verdadero valor de la ingeniería reside en construir sistemas basados en la responsabilidad, confianza y transparencia.

Resumen

En este documento se expone el estudio y desarrollo de una arquitectura de navegación autónoma adaptativa implementada en el simulador CARLA. El sistema propuesto integra un planificador geométrico determinista, basado en el algoritmo A^* , con la capacidad de comprensión semántica del modelo multimodal de visión y lenguaje Llama 4 Scout. Mediante la detección visual de obstáculos en tiempo real y la alteración dinámica de grafos, el vehículo logra recalcular trayectorias para evadir bloqueos urbanos complejos. Se incluye además una evaluación algorítmica rigurosa basada en escenarios personalizados y una métrica matemática de dificultad propia.

Palabras clave: Conducción autónoma, simulador CARLA, modelos multimodales (VLM), planificación dinámica, algoritmo A^* .

Abstract

This document presents the study and development of an adaptive autonomous navigation architecture implemented in the CARLA simulator. The proposed system integrates a deterministic geometric planner, based on the A^* algorithm, with the semantic understanding capabilities of the Llama 4 Scout vision-language multimodal model. Through real-time visual obstacle detection and dynamic graph alteration, the vehicle successfully recalculates trajectories to evade complex urban blockages. Furthermore, it includes a rigorous algorithmic evaluation based on custom scenarios and an original mathematical difficulty metric.

Keywords: Autonomous driving, CARLA simulator, multimodal models (VLM), dynamic planning, A^* algorithm.

Índice

Agradecimientos	vii
Resumen	ix
Abstract	xi
Índice	xiii
Índice de figuras	xvii
Índice de tablas	xix
Índice de algoritmos	xxi
Índice de vídeos	xxiii
1 Introducción	1
1.1 Presentación	1
1.2 Objetivos	2
2 Estudio teórico	3
2.1 Introducción	3
2.2 Estado del Arte	3
2.2.1 Limitaciones de la Navegación Geométrica y Cartográfica	3
2.2.2 Transición hacia la Navegación Semántica	4
2.2.3 Inteligencia Artificial Generativa en la Conducción Autónoma	4
2.2.4 Modelos Multimodales para la Descripción de Entornos	4
2.2.5 Representación del Conocimiento en Navegación	5
3 Desarrollo	7
3.1 Introducción	7
3.2 Arquitectura General del Sistema	8
3.2.1 Módulos Principales	8
3.2.2 Flujo de la Información y Orquestación	8
3.2.2.1 Inicialización del Entorno y Planificación Base	9
3.2.2.2 Adquisición Sensorial y Preprocesamiento Visual	9
3.2.2.3 Razonamiento Semántico y Detección Cognitiva (VLM)	9
3.2.2.4 Traducción Topológica y Re-planificación Dinámica	9
3.2.2.5 Control de Bajo Nivel y Actuación Física	10
3.3 Entorno de Simulación y Modelado de Escenarios	10
3.3.1 Elección y Configuración del Simulador (CARLA)	10

3.3.1.1	Análisis de Plataforma y Versiones	10
3.3.1.2	Adaptación de Mapas con Unreal Engine	11
3.3.2	Metodología de Escenarios Controlados y Medibles	12
3.3.2.1	Selección y Justificación de Entornos	13
3.3.2.2	Generación de Trayectorias y Evaluación Métricas	13
3.4	Subsistema de Navegación y Conducción Autónoma	19
3.4.1	Extracción Topológica y Planificación Global de Trayectorias	19
3.4.1.1	Extracción de la topología en CARLA	19
3.4.1.2	Construcción del grafo navegable	20
3.4.1.3	Planificación Geométrica (Global) mediante Algoritmo A*	21
3.4.2	Módulo de Control y Ejecución	24
3.5	Percepción Multimodal y Replanificación Dinámica	26
3.5.1	Características de Llama 4 Scout en el Contexto de la Conducción Autónoma	26
3.5.2	Proceso de Iteración y Evolución Arquitectónica	29
3.5.3	Arquitectura de Comunicación y Pipeline de Procesamiento	31
3.5.3.1	Captura Sensorial y Preprocesamiento Multi-Vista	32
3.5.3.2	Inferencia Semántica y Sincronización de Estado	32
3.5.3.3	Replanificación Dinámica Online	34
3.5.3.4	Actuación y Bucle de Control Local	36
3.6	Ajustes y Optimización del Modelo Multimodal	37
3.6.0.1	Parametrización de la Inferencia	39
3.6.0.2	Evolución del Contexto Visual y Coste Computacional	39
3.6.0.3	Estrategias de <i>Prompt Engineering</i>	39
3.7	Conclusiones	40
4	Resultados	41
4.1	Introducción	41
4.2	Entorno de Simulación y Condiciones de Prueba	41
4.2.1	Mapas y Topología Vial (CARLA)	41
4.2.2	Mapas y Topología Vial (CARLA)	42
4.2.3	Telemetría y Monitorización (HUD)	42
4.3	Definición de las Métricas de Evaluación	43
4.3.1	Métricas de Percepción y Razonamiento (VLM)	43
4.3.2	Métricas de Planificación y Control	43
4.3.3	Índice de Dificultad del Escenario (M_{total})	44
4.4	Diseño y Justificación del Catálogo de Pruebas	44
4.5	Análisis Global de Resultados Experimentales	45
4.5.1	Navegación en Entornos de Baja y Media Complejidad	45
4.5.2	Anomalías Perceptuales y Falsos Positivos Sistémicos	45
4.5.3	Limitaciones en Geometrías Restrictivas y Casos de Colisión	46
4.6	Conclusiones	46
5	Conclusiones y líneas futuras	49
5.1	Conclusiones	49
5.1.1	Evaluación de Objetivos y Viabilidad del Sistema	49
5.1.2	Contribuciones Tecnológicas a la Arquitectura de Navegación	49
5.1.3	Análisis de Robustez y Limitaciones Operativas	50

5.2	Líneas futuras	50
5.2.1	Optimización del Módulo de Percepción y Edge AI	50
5.2.2	Refinamiento de la Capa de Planificación, Grafos Dinámicos y Control Predictivo	51
5.2.3	Ampliación del Marco de Validación y Escenarios Cooperativos	51
Bibliografía		53
Apéndice A Manual de usuario		57
A.1	Introducción	57
A.2	Requisitos previos e Instalación	57
A.2.1	Software base necesario	57
A.2.2	Instalación de dependencias	58
A.2.3	Configuración de la clave de la API de Groq	58
A.3	Ejecución de la Simulación	58
A.3.1	Arranque del sistema	58
A.3.2	Menú interactivo de configuración	59
A.3.3	Interacción en tiempo real (Override Manual)	59
A.4	Catálogo de Pruebas y Demostraciones Visuales	59
Apéndice B Herramientas y tecnologías utilizadas		61
B.0.1	Hardware	61
B.0.2	Entorno y Sistemas Operativos	61
B.0.3	Lenguaje de Programación	61
B.0.4	Bibliotecas y Módulos de Python	62
B.0.4.1	Visión Artificial y Procesamiento de Datos	62
B.0.4.2	Planificación de Rutas y Grafos	62
B.0.4.3	Manejo de Archivos y Concurrencia	62
B.0.5	Inteligencia Artificial y Modelos Multimodales (VLM)	62
B.0.6	Herramientas de Documentación y Control de Pruebas	63
B.0.7	Control de Versiones y Gestión del Proyecto	63

Índice de figuras

1.1	Arquitectura modular de lazo cerrado del sistema adaptativo de navegación autónoma. . .	2
3.1	Vistas del entorno Town 02. (a) Vista aérea general del mapa. (b) Vista detallada de la calle principal (High Street).	14
3.2	Vistas del entorno Town 03. (a) Vista aérea general del mapa. (b) Vista detallada de la rotonda.	15
3.3	Vistas del entorno Town 04. (a) Vista aérea general del mapa. (b) Vista detallada del paso elevado (overpass).	16
3.4	Vistas del entorno Town 10. (a) Vista aérea general del mapa. (b) Vista detallada de la intersección (junction).	17
3.5	Representación visual de la topología de una intersección en CARLA. Las líneas continuas (amarillo) ilustran las conexiones lógicas directas entre vías convergentes (pares de entrada-salida del objeto <code>carla.Junction</code>), mientras que la discretización puntual (azul) muestra la trayectoria interpolada generada por el <code>GlobalRoutePlanner</code>	20
3.6	Comparativa topológica del entorno de simulación. (a) Vista aérea tridimensional del mapa físico en CARLA. (b) Representación espacial bidimensional del grafo de navegación dirigido $G = (V, E)$ extraído de dicha topología. Se aprecia la correspondencia directa entre los cruces físicos y la densidad de vértices y aristas en la abstracción matemática. .	21
3.7	Asignación de maniobras (<i>RoadOption</i>) en CARLA. El producto vectorial de las trayectorias de entrada ($\vec{v}_{in}$) y salida ($\vec{v}_{out}$) define el giro: (a) giro a la izquierda para $w_z > 0$, y (b) giro a la derecha para $w_z < 0$	24
3.8	Arquitectura de control jerárquico del subsistema de navegación autónoma y lazo de interacción con el entorno de simulación CARLA.	27
3.9	Diagrama de bloques de la arquitectura de sistemas de Llama 4 Scout. Se ilustra la ingesta multimodal en paralelo y la convergencia de datos en el mecanismo de Mezcla de Expertos (MoE).	28
3.10	Esquema geométrico del preprocesamiento de la imagen (<i>Region of Interest</i>). En el eje vertical, se aplica un recorte entre el 20% y el 80% para descartar el cielo y el posible chasis del vehículo o asfalto inmediato. En el eje horizontal, las tres vistas mantienen un solapamiento del 10% para garantizar la conservación del contexto espacial en las transiciones visuales.	31

3.11	Interfaz de visualización (HUD) en CARLA. En el panel superior izquierdo se monitoriza la telemetría del vehículo junto con los parámetros del modelo multimodal. Los paneles derecho e inferior exponen el historial y la salida actual del razonamiento semántico de la IA.	31
3.12	Diagrama de Autómata de Estados Finitos (FSM) que rige el ciclo de vida concurrente del hilo de percepción <code>vlm_worker</code>	34
3.13	Diagrama de secuencia del pipeline concurrente. El hilo principal mantiene la ejecución de físicas (<code>world.tick</code>) sin bloqueos mientras el hilo secundario gestiona la latencia de red con la API remota.	38

Índice de tablas

3.1	Notas sobre la compatibilidad entre versiones 0.9.x de CARLA.	11
3.2	Resumen de los mapas (Towns) disponibles en CARLA.	12

Índice de algoritmos

3.1	Alteración Dinámica del Espacio de Costes	23
3.2	Mecanismo de Histéresis Espacial y Búsqueda de Waypoint (Look-ahead)	36
3.3	Bucle de Control Local y Sincronización Temporal Dinámica	38

Índice de vídeos

A.1	TC-TWN02-004: Variante topológica con múltiples obstáculos sucesivos (enlace al vídeo)	60
A.2	TC-TWN03-003: Intervención multisectorial de alta complejidad (bomberos, obras y policía) (enlace al vídeo)	60
A.3	TC-TWN04-002: Identificación de balizamiento mimetizado frente a ruido visual de fondo (enlace al vídeo)	60
A.4	TC-TWN04-009: Detección de trampa topológica profunda y ejecución de maniobra de retorno (enlace al vídeo)	60
A.5	TC-TWN10HD-004: Triple bloqueo con detección crítica de obstáculo escorado en curva (enlace al vídeo)	60
A.6	TC-TWN10HD-008: Operación policial bajo condiciones visuales adversas (sombras y baja iluminación) (enlace al vídeo)	60

Capítulo 1

Introducción

En este primer capítulo se expone la motivación y el contexto que justifican el desarrollo del presente Trabajo de Fin de Grado, así como los objetivos principales y específicos que se persiguen con su realización.

1.1 Presentación

En la actualidad, la conducción autónoma representa uno de los mayores desafíos tecnológicos y científicos. La capacidad de un vehículo para desplazarse de forma independiente y segura depende directamente de su habilidad para percibir el entorno, interpretar la información sensorial y tomar decisiones complejas en fracciones de segundo.

Tradicionalmente, los sistemas de conducción autónoma se han apoyado fuertemente en el paradigma de la navegación geométrica, utilizando Sistemas Globales de Navegación por Satélite (GNSS) y Mapas de Alta Definición (HD Maps). Sin embargo, los entornos viales urbanos son altamente dinámicos e inciertos. La presencia de obras inesperadas, accidentes, vehículos de emergencia o alteraciones temporales en la vía deja en evidencia la vulnerabilidad de depender exclusivamente de mapas estáticos y coordenadas predefinidas. Cuando el entorno físico difiere del mapa teórico, los planificadores puramente geométricos tienden a fallar, provocando bloqueos del sistema o decisiones de navegación inseguras.

Es por ello que la industria y la investigación están transitando hacia una navegación semántica, donde el vehículo no solo mida distancias, sino que "comprenda" el contexto que le rodea de una forma similar a como lo haría un conductor humano. En esta transición, la Inteligencia Artificial Generativa y, más concretamente, los Modelos Multimodales de Visión y Lenguaje (VLM, por sus siglas en inglés), han irrumpido como una herramienta revolucionaria. Modelos como Llama 4 Scout poseen una capacidad excepcional para procesar imágenes en tiempo real y emitir razonamientos lógicos sobre la viabilidad del espacio transitable.

No obstante, estos modelos multimodales tienen una naturaleza probabilística y carecen de un arraigo espacial tridimensional exacto, lo que en ocasiones puede derivar en falsos positivos visuales (alucinaciones) al confundir el mobiliario urbano periférico con obstáculos reales en la calzada. Por lo tanto, no pueden asumir el control físico directo del vehículo sin comprometer la seguridad.

En este contexto nace la motivación del presente trabajo: la necesidad de crear una arquitectura híbrida y modular que una "lo mejor de ambos mundos". Por un lado, la capacidad de abstracción y comprensión contextual de los grandes modelos multimodales para detectar obstáculos y anomalías; y

por otro, la precisión, seguridad y robustez de los algoritmos de planificación geométrica determinista. Para diseñar, entrenar y validar de forma segura arquitecturas de esta complejidad, el uso de entornos de simulación de alta fidelidad como CARLA resulta completamente imprescindible, ya que permite recrear el dinamismo del tráfico real en un ecosistema controlado y medible.

1.2 Objetivos

El objetivo principal de este trabajo consiste en desarrollar un sistema de planificación geométrica de trayectorias en el simulador CARLA, capaz de calcular rutas entre puntos de origen y destino en entornos urbanos y adaptarse dinámicamente a las alteraciones de la vía. Para lograrlo, desde una perspectiva aplicada, se diseñarán y personalizarán escenarios de prueba, lo que permitirá evaluar el comportamiento del sistema bajo diferentes topologías y niveles de complejidad.

Sobre este entorno simulado se integrará un módulo de percepción visual que empleará modelos multimodales para procesar las imágenes de las cámaras del vehículo. Este componente tendrá el propósito de generar descripciones semánticas comprensibles del entorno de manera específica al detectar un obstáculo o anomalía en la calzada, traduciendo esta evaluación cualitativa en comandos que permitan al planificador modificar el grafo topológico en tiempo real y trazar rutas alternativas seguras.

Finalmente, la validación del trabajo abarcará la evaluación integral de la precisión perceptiva y la robustez geométrica del sistema frente a estos imprevistos. De este modo, la implementación propuesta no solo aborda el desafío técnico de la navegación autónoma, sino que establece las bases para crear interfaces de supervisión más intuitivas y sistemas de validación más accesibles, elementos esenciales para la futura adopción de esta tecnología en entornos urbanos reales.

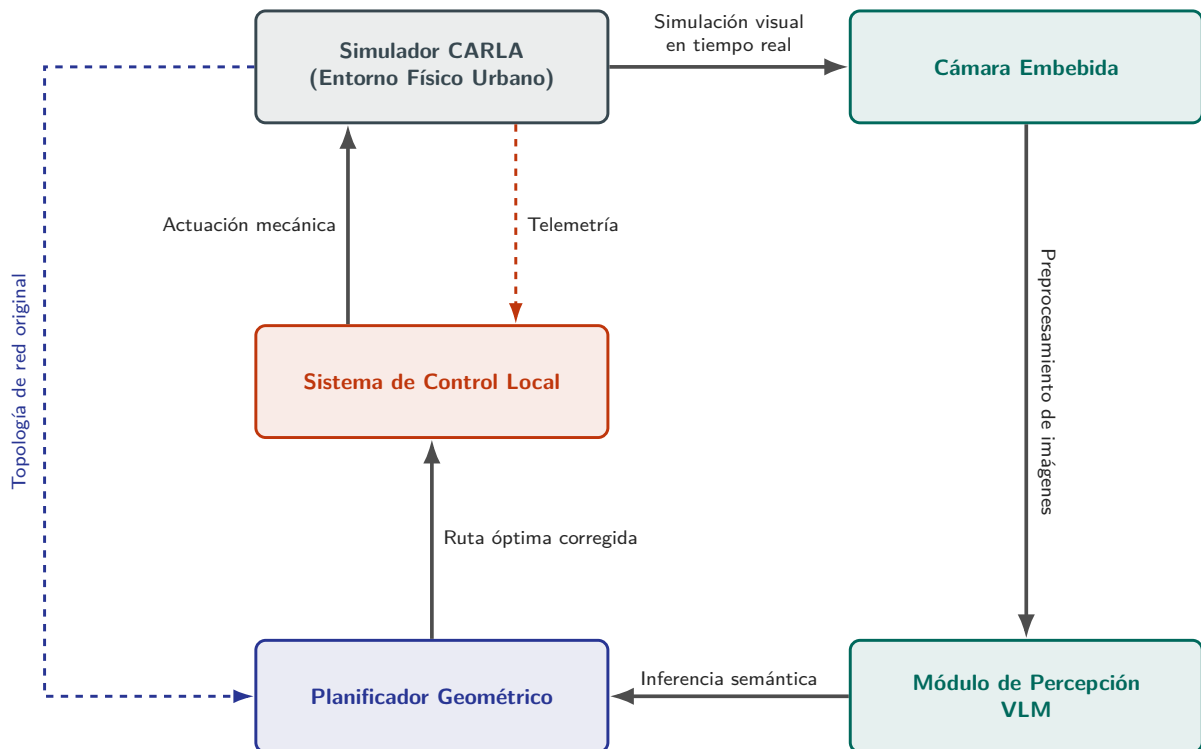


Figura 1.1: Arquitectura modular de lazo cerrado del sistema adaptativo de navegación autónoma.

Capítulo 2

Estudio teórico

2.1 Introducción

El presente capítulo tiene como objetivo establecer el marco conceptual, teórico y metodológico que fundamenta el desarrollo de este trabajo. En el escenario actual de la conducción autónoma, la consecución de una navegación robusta, adaptativa y segura exige transcender los enfoques puramente mecánicos o geométricos, transitando hacia modelos capaces de interpretar semánticamente el entorno y razonar de manera lógica ante la incertidumbre del tráfico real. Por ello, este espacio se dedica a examinar de forma crítica la literatura científica reciente, delimitando las fronteras del conocimiento e identificando las brechas técnicas que justifican la propuesta arquitectónica implementada en este proyecto.

A través de este recorrido teórico, se consolidan los fundamentos necesarios para justificar las decisiones de diseño, control y simulación que se detallarán en los capítulos subsiguientes.

2.2 Estado del Arte

2.2.1 Limitaciones de la Navegación Geométrica y Cartográfica

El desarrollo histórico de los sistemas de conducción autónoma se ha fundamentado en el paradigma de la navegación geométrica, una arquitectura de control clásico (Sense-Plan-Act) que conceptualiza el mundo físico como un espacio euclidiano [1]. La capacidad de un vehículo para trazar una ruta depende de dos componentes tecnológicos: los Sistemas Globales de Navegación por Satélite (GNSS) para la posición absoluta, y los Mapas de Alta Definición (HD Maps) para estructurar el grafo de los algoritmos de planificación como A^* o Dijkstra. Sin embargo, en entornos urbanos densos, el GNSS sufre degradaciones severas por efecto multitrayecto y falta de visibilidad [2]. Esto obliga a depender de la odometría y algoritmos SLAM, introduciendo un error de deriva acumulativo que desorienta espacialmente a los planificadores geométricos al perder su marco de referencia exacto [3].

Para mitigar esta inestabilidad, la industria consolidó el uso de HD Maps como redundancia espacial, incluyendo capas topológicas, geométricas y semánticas precisas. No obstante, la literatura demuestra que su naturaleza estática constituye una vulnerabilidad estructural crítica [4]. Sato et al. (2025)[5] exponen cómo anomalías topológicas menores introducen fallos sistémicos en la navegación. Dado el alto coste de actualización de estos mapas, sufren invariablemente de latencia temporal respecto al mundo físico dinámico (obras, accidentes, desvíos). Esta discrepancia genera una sobredependencia cartográfica: ante la divergencia entre el grafo teórico y la realidad captada por los sensores, el planificador

se bloquea o fuerza trayectorias peligrosas. Además, advierten[5] sobre la vulnerabilidad ante ataques de alteración de vectores cartográficos, demostrando empíricamente que la dependencia de mapas inmutables es insuficiente para una conducción robusta y segura.

2.2.2 Transición hacia la Navegación Semántica

Para superar las limitaciones de la localización pura por coordenadas, la investigación ha propiciado un cambio de paradigma hacia la navegación semántica. Esta metodología emula los procesos cognitivos humanos, abstrayendo el espacio de conducción para priorizar la comprensión relacional del entorno frente a su medición kilométrica. Un planificador semántico procesa directrices relacionales y adapta dinámicamente su entendimiento de la vía, abstrayendo obstáculos imprevistos para activar heurísticas superiores que luego se delegan al controlador geométrico de bajo nivel.

Durante esta transición, gran parte de la investigación adoptó arquitecturas de "Extremo a Extremo" (E2E) mediante clonación de comportamiento, condensando percepción y planificación en una única red neuronal. Sin embargo, al operar como "cajas negras", imposibilitan la auditoría de seguridad. Las arquitecturas E2E sufren de "confusión causal"[6], identificando falsas correlaciones estadísticas en vez de relaciones causales reales, lo que en simulación de lazo cerrado desencadena un error compuesto ante mínimas perturbaciones. Esta debilidad estructural justifica la necesidad de adoptar enfoques de navegación modular interpretables.

2.2.3 Inteligencia Artificial Generativa en la Conducción Autónoma

Ante los fallos deductivos de los enfoques E2E, la investigación converge hacia la Inteligencia Artificial Generativa, integrando Grandes Modelos de Lenguaje (LLMs) como emuladores de sentido común para resolver escenarios de tráfico inciertos. Chen et al. (2024)[7] demuestran que, al preprocesar el entorno mediante una modalidad espacial vectorial de vista de pájaro (BEV), el LLM puede interpretar contextos viales y emitir comandos de control acompañados de justificaciones semánticas. Llevando esta capacidad al nivel operativo, el marco LaMPilot[8] evidencia cómo los LLMs actúan como generadores de código en tiempo real, compilando instrucciones en lenguaje natural en políticas de conducción Python ejecutables mediante APIs del vehículo.

Para solventar la inviabilidad de ejecutar modelos masivos directamente en las Unidades de Control Electrónico (ECU) del vehículo por latencia y memoria, técnicas de Ajuste Fino Eficiente (PEFT/LoRA) [9], [10] han logrado especializar los pesos de los LLMs para tareas explícitas de autopista, logrando comportamientos preventivos en tiempo real[11]. Sin embargo, persiste un obstáculo arquitectónico crítico: la base matemática de estos modelos generativos es probabilística (predicción estocástica de tokens), lo que imposibilita garantizar la completitud lógica y la verificabilidad formal en casos extremos. Por ello, las salidas de un modelo fundacional requieren ineludiblemente ser convalidadas por un planificador determinista antes de alterar el movimiento físico del automóvil.

2.2.4 Modelos Multimodales para la Descripción de Entornos

Para que la planificación semántica sea posible, es indispensable transformar la información visual en bruto (píxeles) en representaciones estructuradas, tarea abordada por los Modelos de Visión-Lenguaje (VLMs). Arquitecturas recientes como DriveVLM-Dual[12] se encargan de gestionar los eventos de la "larga cola" hibridando la deducción semántica del modelo multimodal con la precisión geométrica de los sensores LiDAR, demostrando que la cognición semántica debe actuar como orquestadora sobre los

planificadores tradicionales. Modelos fundacionales como Llama-4 Scout optimizan este proceso mediante fusión temprana de modalidades, disminuyendo los requisitos de VRAM para habilitar su ejecución nativa.

Pese a los avances, la ingesta en tiempo real de flujos de vídeo por VLMs masivos en el bucle de control choca con estrictas limitaciones temporales. Investigaciones como ThinkDrive[13] alertan que ubicar la inferencia semántica en la ruta crítica detiene al planificador local, forzando arquitecturas asíncronas de doble proceso. Asimismo, los VLMs carecen de un arraigo espacial tridimensional inherente, lo que genera recurrentes alucinaciones perceptuales: catalogar reflejos o sombras como amenazas físicas inminentes, provocando el bloqueo del sistema. Esta vulnerabilidad perceptual demuestra la necesidad de combinar el análisis semántico con un planificador geométrico robusto para descartar falsos positivos.

2.2.5 Representación del Conocimiento en Navegación

Para aprovechar la información de los VLM sin heredar sus alucinaciones, la solución pasa por integrar marcos de Representación del Conocimiento y Razonamiento (Knowledge Representation and Reasoning, KRR). En este sentido, HUMANA[14] propone un enfoque novedoso: emplear los modelos generativos no para el control directo, sino como traductores que formulan la escena en reglas de Answer Set Programming (ASP). Esta lógica declarativa no monótona combina las normas de tráfico inmutables con las condiciones dinámicas del entorno descritas por el LLM en tiempo real. El resultado son trayectorias verificables (de caja blanca) y la capacidad de corregir decisiones cuando la información de los sensores cambia.

Sin embargo, en la práctica, este sistema presenta una limitación metodológica: tanto el motor lógico como el algoritmo geométrico (A^*)[15] operan aislados, ya que carecen de percepción directa para reaccionar a los cambios físicos del entorno. Aquí reside la principal aportación de este proyecto: integrar el planificador A^* con una capa perceptiva controlada por un modelo multimodal avanzado. El VLM procesa el entorno y envía información dinámica actualizada al motor. Al filtrar el ruido visual y actualizar los costes espaciales, el algoritmo A^* recalcula trayectorias adaptadas al estado real del entorno en cada instante. Esta integración justifica la arquitectura implementada en los simuladores CARLA [16], [17] y Unreal Engine [18] que se detalla en el próximo capítulo.

Capítulo 3

Desarrollo

3.1 Introducción

En este capítulo se expone en detalle el desarrollo técnico y metodológico del trabajo, estructurando la implementación de un sistema de conducción autónoma fundamentado en una arquitectura modular, distribuida y de lazo cerrado. El propósito principal de este diseño es evaluar escenarios de conducción urbana mediante la integración de razonamiento semántico avanzado y planificación determinista, segregando las responsabilidades en componentes especializados para evitar la dependencia de modelos monolíticos. Para facilitar la comprensión del diseño global, la integración tecnológica y los algoritmos implementados, el contenido de este capítulo se ha organizado siguiendo una secuencia lógica orientada a subsistemas.

En primer lugar, se describe la arquitectura general del sistema, definiendo el rol específico de sus módulos principales: el entorno de simulación CARLA, el modelo multimodal Llama-4 Scout, el planificador geométrico y el sistema de control de conducción, así como la orquestación del flujo de la información. Posteriormente, se profundiza en el entorno de simulación y el modelado de escenarios controlados. En dicho apartado se justifica la infraestructura tecnológica empleada, abarcando la adaptación de mapas mediante el motor gráfico Unreal Engine, y se formula matemáticamente una métrica de dificultad global objetiva para evaluar la exigencia analítica de las trayectorias de validación.

A continuación, el documento aborda el subsistema de navegación y conducción autónoma, explicando detalladamente la extracción topológica del espacio transitable basada en el estándar OpenDRIVE, la construcción de un grafo navegable y la consecuente planificación geométrica mediante el algoritmo heurístico A*. En este mismo contexto, se expone el módulo de control y ejecución, encargado de traducir las trayectorias lógicas en comandos mecánicos mediante controladores PID gestionados por la arquitectura del BehaviorAgent. Seguidamente, se presenta el núcleo de percepción multimodal y replanificación dinámica, el cual integra el Modelo de Visión-Lenguaje (VLM) Llama-4 Scout en sustitución de las redes de segmentación clásicas. Esta sección documenta la evolución hacia un procesamiento multi-vista para mitigar la pérdida de contexto lateral, el diseño de un pipeline concurrente para estabilizar la latencia temporal, y el mecanismo de replanificación fundamentado en histéresis espacial.

Finalmente, se detallan los ajustes y la optimización empírica del modelo multimodal, abordando los retos técnicos inherentes a la integración de inteligencia artificial generativa dentro de un entorno físico simulado. En este epígrafe se describen la parametrización determinista de la inferencia y la aplicación de estrategias de *Prompt Engineering*, tales como el *Negative Prompting* y el *Chain-of-Thought*. Estas

técnicas resultan fundamentales para corregir las deficiencias de razonamiento espacial tridimensional del modelo y dotar de robustez al planificador frente a las complejidades del entorno urbano.

3.2 Arquitectura General del Sistema

El desarrollo de este proyecto se fundamenta en una arquitectura modular, distribuida y de lazo cerrado, diseñada para evaluar escenarios de conducción autónoma mediante la integración de razonamiento semántico avanzado y planificación determinista. En lugar de depender de un único modelo monolítico que resuelva la conducción de extremo a extremo, el sistema segrega las responsabilidades en componentes especializados que se comunican de forma continua.

Esta separación de tareas permite que el sistema sea escalable, interpretable y robusto frente a cambios en el entorno urbano. A continuación, se describen los módulos principales que conforman la arquitectura y el rol específico que desempeña cada uno dentro del ecosistema global.

3.2.1 Módulos Principales

- Entorno de Simulación (CARLA): actúa como la base del proyecto. CARLA es el motor que proporciona un entorno físico 3D altamente realista, gestionando la cinemática del vehículo, las colisiones y el comportamiento de otros agentes del tráfico. Además, actúa como el proveedor principal de los datos sensoriales del vehículo, generando específicamente las transmisiones de video (imágenes RGB) necesarias para la percepción del entorno.
- Modelo Multimodal (Llama-4 Scout): representa el núcleo de razonamiento cognitivo y semántico de la arquitectura. Este módulo actúa como una red neuronal de alto nivel que procesa las imágenes capturadas por las cámaras del vehículo simulado. Su función no es calcular trayectorias, sino interpretar el contexto de la escena de forma análoga a la intuición humana (por ejemplo, detectar si un carril está bloqueado por obras o si existen obstáculos críticos en la calzada).
- Planificador Geométrico: es el componente algorítmico encargado de la viabilidad espacial y la navegación a largo plazo. Utiliza algoritmos de búsqueda sobre grafos (como A*) para trazar la ruta óptima desde un punto de origen hasta un destino mediante una sucesión de waypoints. Este módulo es altamente dinámico; posee la capacidad de adaptar los pesos topológicos del mapa en tiempo real para recalcular trayectorias alternativas cuando recibe notificaciones de bloqueo.
- Sistema de Control de Conducción: funciona como el puente entre las decisiones algorítmicas de alto nivel y los actuadores físicos del coche. Este módulo traduce la ruta geométrica en comandos de control de bajo nivel: aceleración, freno y ángulo de dirección. Asimismo, incorpora lógicas reactivas de seguridad, como el mantenimiento de la distancia con el vehículo precedente y la parada de emergencia ante semáforos o peatones.

3.2.2 Flujo de la Información y Orquestación

La verdadera capacidad del sistema reside en la orquestación continua de sus módulos mediante un bucle de retroalimentación de lazo cerrado, que actualiza el estado del vehículo en cada instante (tick) de la simulación. El procesamiento de la información sigue una secuencia lógica y estructurada que abarca desde la adquisición de datos hasta la actuación física:

3.2.2.1 Inicialización del Entorno y Planificación Base

El ciclo de vida del sistema arranca con la lectura de los archivos de configuración (en formato YAML), los cuales definen las coordenadas geográficas de inicio y destino, así como la disposición de posibles obstáculos en la simulación. Una vez instanciado el mapa en CARLA, el Planificador Geométrico (GlobalRoutePlanner) [19] analiza la red vial y construye una representación matemática del entorno en forma de grafo dirigido (NetworkX), donde los nodos son coordenadas y las aristas representan los carriles navegables. Sobre este grafo base, el algoritmo de búsqueda A* traza de forma determinista la trayectoria óptima inicial para conectar el punto de aparición con el destino.

3.2.2.2 Adquisición Sensorial y Preprocesamiento Visual

Una vez que el vehículo inicia su marcha, el simulador CARLA asume el rol de proveedor de datos sensoriales. A través de una cámara RGB frontal acoplada al chasis (configurada con un amplio campo de visión), el sistema captura el entorno en tiempo real. Para optimizar el análisis y dotar al modelo de inteligencia artificial de un mayor contexto espacial, cada imagen capturada (el frame completo) se segmenta mediante técnicas de visión artificial en tres recortes superpuestos: vista izquierda, central y derecha. Este preprocesamiento asegura que el sistema pueda evaluar simultáneamente tanto el carril frontal por el que circula como las posibles vías de escape en las intersecciones adyacentes.

3.2.2.3 Razonamiento Semántico y Detección Cognitiva (VLM)

Las tres vistas generadas se envían de forma asíncrona al Modelo Multimodal (Llama-4 Scout). Este componente actúa como el cerebro cognitivo de la arquitectura, evaluando la escena bajo directrices estrictas: debe discriminar entre elementos inofensivos ubicados fuera de la calzada (como aceras, farolas o peatones estáticos) y obstáculos críticos que bloquean físicamente el asfalto navegable. El resultado de esta inferencia es un evento estructurado en formato JSON que especifica si existe un bloqueo temporal, la posición relativa del mismo (izquierda, centro o derecha), su nivel de confianza y una descripción en lenguaje natural del razonamiento aplicado. Para optimizar los recursos, estas justificaciones se almacenan en un historial temporal, evitando evaluaciones redundantes en fotogramas sucesivos si el contexto no ha cambiado sustancialmente.

3.2.2.4 Traducción Topológica y Re-planificación Dinámica

Si el modelo detecta una vía bloqueada, el flujo transfiere el evento semántico al gestor de re-planificación (ReplanningManager). Este submódulo traduce la orden abstracta de la IA (por ejemplo, "cruce a la derecha bloqueado") en coordenadas matemáticas concretas, identificando los waypoints exactos de la intersección afectada en el mapa de CARLA. Inmediatamente después, el gestor de penalizaciones (RoutePenaltyManager) interviene modificando dinámicamente el grafo de navegación: aplica un coste de travesía prohibitivamente alto (penalización) a las aristas correspondientes a la zona bloqueada. Con el mapa actualizado, el sistema invoca nuevamente al algoritmo de búsqueda, forzándolo a calcular una ruta alternativa inmediata partiendo desde la ubicación actual del vehículo. Adicionalmente, el sistema registra una "burbuja de exclusión" de seguridad (ej. 30 metros) alrededor del evento para no volver a evaluar la misma intersección mientras el vehículo la esquivaba.

3.2.2.5 Control de Bajo Nivel y Actuación Física

Con la nueva ruta geométrica asegurada, el flujo concluye en el Sistema de Control de Conducción (BehaviorAgent y RouteManager), responsable de la dinámica longitudinal y lateral del vehículo. A nivel lateral, se calcula el ángulo de giro óptimo del volante analizando los waypoints futuros (mediante técnicas de lookahead) para mantener el coche centrado en el carril. A nivel longitudinal, el agente supervisa factores dinámicos del entorno a corto alcance: regula la velocidad máxima permitida, frena ante semáforos en rojo, mantiene la distancia de seguridad con vehículos precedentes y activa detenciones de emergencia si peatones irrumpen en su zona de frenado. Estos cálculos se empaquetan finalmente en un objeto de control (carla.VehicleControl de aceleración, freno y dirección) que se inyecta de vuelta al simulador CARLA para materializar el movimiento del vehículo, reiniciando así el ciclo.

3.3 Entorno de Simulación y Modelado de Escenarios

Esta sección detalla la infraestructura tecnológica y la base metodológica sobre la cual se sustentan las pruebas experimentales de este proyecto. El desarrollo de los escenarios de validación requirió una cuidadosa selección de herramientas, la integración de motores gráficos y la formulación de métricas objetivas para evaluar el comportamiento en entornos urbanos simulados.

3.3.1 Elección y Configuración del Simulador (CARLA)

El desarrollo experimental de este proyecto exige un entorno de simulación capaz de proveer físicas realistas, flexibilidad en la alteración de mapas y una integración fluida con arquitecturas de software para la toma de decisiones. Bajo estas premisas, se ha seleccionado CARLA como la herramienta central del entorno de simulación. CARLA es un simulador de código abierto diseñado específicamente para la investigación en conducción autónoma, el cual proporciona un control total sobre el ecosistema mediante una robusta API en Python y C++.

3.3.1.1 Análisis de Plataforma y Versiones

Durante la fase inicial de conceptualización, se llevó a cabo un estudio técnico comparativo para determinar el entorno de despliegue óptimo y la versión del simulador más adecuada para los objetivos del proyecto.

En cuanto al sistema operativo, aunque CARLA ofrece compatibilidad tanto con Windows (10/11) como con distribuciones Linux (Ubuntu 20.04/22.04), se constató que la infraestructura de desarrollo presenta un mejor soporte oficial en entornos Linux. En Linux, la compilación desde el código fuente resulta más estable, el rendimiento de los fotogramas por segundo (FPS) tiende a ser superior y facilita la automatización mediante herramientas de despliegue como Docker.

Respecto a la versión del simulador, se analizó el salto tecnológico entre la rama estable 0.9.x y la reciente versión 0.10.0. Si bien la versión 0.10.0 incorpora mejoras visuales sustanciales al migrar al motor Unreal Engine 5, esta actualización presenta limitaciones críticas para el flujo de trabajo del proyecto, al carecer de soporte para la importación nativa de OpenDRIVE (.xodr), carecer de integración con datos de OpenStreetMap, y no soportar la API de capas de mapas (Map Layers API). Por consiguiente, se determinó el uso de la rama 0.9.x (específicamente iterando entre las versiones 0.9.14 y 0.9.16), las cuales mantienen una API estable sin cambios significativos entre ellas y soportan de forma nativa todas las tecnologías de alteración de redes viales requeridas.

Tabla 3.1: Notas sobre la compatibilidad entre versiones 0.9.x de CARLA.

Versión	Año	Mapas / escala	Sensores & visión	API / usabilidad	Notas prácticas
0.9.0	2018	Mapas clásicos	Sensor básico	Nueva Python API; multi-cliente/agent. Ideal para empezar a scriptar. (CARLA Simulator)	Punto de partida para migraciones desde versiones 0.8.x
0.9.11	2020	Mapas mejorados	Más blueprints	Control por capas en mapas (activar/desactivar objetos). Útil para pruebas ablativas. (CARLA Simulator)	Mejor para pruebas físicas más realistas
0.9.12	2021	Large maps (Town11 demo 400 km ² ; soporte a escala)	Optical Flow Camera; más vehículos/peatones	Instalación pip para client; off-screen rendering posible por UE4.26. (CARLA Simulator)	Excelente para datasets a gran escala y pipelines deterministas
0.9.13	2021	Large maps continuadas	Instance segmentation, runtime texture streaming, 3D skeleton GT	Traffic Manager mejorado; más assets. (CARLA Simulator)	Ideal para investigación en percepción (segm, skeleton)
0.9.14	2022	Town12: ~10×10 km ² mapa grande con alto detalle	Mejores assets y atributos vehículos; gbuffer APIs	Acceso directo a texturas GBuffer (listen_to_gbuffer), de GBuffer y más opciones para weather param tweaks. (CARLA Simulator)	Para visualización avanzada, captura de GBuffer y generación de datos sintéticos
0.9.15	2023	Nuevas herramientas para procedurales	Herramientas de procedural map & building generation; nuevos vehículos (HGV)	Utilities para acelerar creación de mapas; experimental. (CARLA Simulator)	Mejor para crear variedad procedimental y tests de generalización

3.3.1.2 Adaptación de Mapas con Unreal Engine

El fotorrealismo y la gestión de colisiones físicas en CARLA recaen sobre el motor gráfico Unreal Engine. Los entornos de conducción en CARLA no son archivos únicos, sino que se componen de dos capas principales que interactúan estrechamente: una descripción lógica de la red vial en formato estándar .xodr (OpenDRIVE) y un conjunto de recursos tridimensionales (.fbx, materiales y texturas) procesados por Unreal Engine.

Para el diseño y comprensión de la topología de la vía, se incluyó en el flujo de trabajo el software MathWorks RoadRunner. Este software permite estructurar intersecciones complejas y exportar escenarios en formato OpenDRIVE compatibles con el importador de CARLA.

No obstante, para introducir alteraciones viales representativas (obras, calles deterioradas o bloqueos de policía), se optó por aprovechar el ecosistema de assets de Unreal Engine. En lugar de alterar la malla asfáltica lógica, la adaptación del mapa se realizó incorporando props [20] físicos sobre el entorno original. Debido a que algunos objetos decorativos de Unreal Engine no están expuestos por defecto al cliente de CARLA, fue necesario modificar el archivo de registro interno Default.Package.json. Esta modificación instruye a Unreal Engine para que compile Blueprints válidos de dichos recursos estáticos, permitiendo que la API de Python [19] los reconozca e instancie en tiempo de ejecución, dotándolos de un volumen de colisión que el planificador de rutas del agente debe respetar forzosamente.

3.3.2 Metodología de Escenarios Controlados y Medibles

La validación de trayectorias requirió un proceso sistemático para definir escenarios de prueba. Inicialmente, se categorizaron los mapas base disponibles para descartar aquellos con poca utilidad práctica. A continuación se muestran todos los mapas disponibles en la versión 0.9.16 de CARLA en la tabla 3.2.

Tabla 3.2: Resumen de los mapas (Towns) disponibles en CARLA.

Town	Resumen
Town01	Una pequeña y sencilla ciudad con un río y varios puentes.
Town02	Una pequeña y sencilla ciudad con una mezcla de edificios residenciales y comerciales.
Town03	Un mapa urbano más grande con una rotonda y grandes cruces.
Town04	Una pequeña ciudad enclavada en las montañas con una carretera infinita con una forma especial de «8».
Town05	Ciudad con cuadrícula cuadrada, cruces y un puente. Tiene varios carriles por sentido. Útil para realizar cambios de carril.
Town06	Autopistas largas de varios carriles con muchas entradas y salidas. También tiene un giro a la izquierda tipo Michigan.
Town07	Un entorno rural con carreteras estrechas, maíz, graneros y apenas semáforos.
Town08	Pueblo secreto «invisible» utilizado para el desafío de la tabla de clasificación.
Town09	Pueblo secreto «invisible» utilizado para el desafío de la tabla de clasificación.
Town10	Un entorno urbano céntrico con rascacielos, edificios residenciales y un paseo marítimo.
Town11	Un mapa grande sin decorar. Sirve como prueba de concepto para la función de mapas grandes.
Town12	Un mapa grande con numerosas regiones diferentes, incluyendo entornos de rascacielos, residenciales y rurales.

3.3.2.1 Selección y Justificación de Entornos

Se seleccionó un conjunto de mapas para cubrir un espectro gradual de complejidad de la conducción autónoma:

- **Town02 (Pruebas Básicas y Datos Limpios):** como se aprecia en la Figura 3.1, este entorno de dimensiones reducidas se seleccionó para la generación de las trayectorias más fundamentales del dataset. Al estar compuesto por cruces sencillos (principalmente intersecciones en "T"), permite evaluar la toma de decisiones del agente en situaciones de baja complejidad, garantizando un punto de partida estable para las validaciones iniciales.
- **Town03 (Diversidad Topológica y Rotondas):** este mapa (véase la Figura 3.2) destaca por incluir una rotonda central de dos carriles estándar, cruces asimétricos (en "T" y "+") y un carril de incorporación a la circulación desde una gasolinera. Su alta densidad de agentes semánticos (semáforos, señales de Stop, límites de velocidad y pasos de peatones) lo convierte en un escenario idóneo para probar maniobras de navegación complejas.
- **Town04 (Alta Velocidad y Larga Distancia):** la inclusión de este mapa, ilustrado en la Figura 3.3, se justifica por su topología en forma de "8", la cual emula una carretera o autopista de alta capacidad. Sus características resultan relevantes para la evaluación de trayectorias largas y continuas, así como para analizar el comportamiento del vehículo en carriles de aceleración, entradas y salidas a altas velocidades.
- **Town10HD (Alta Densidad Semántica y Entorno Urbano):** mostrado detalladamente en la Figura 3.4, este mapa recrea un entorno céntrico de alta fidelidad visual y semántica. Se seleccionó por albergar la mayor concentración y variedad de elementos urbanos (marcas viales complejas, paradas de autobús, diversidad de señales verticales y rascacielos). Representa el mayor grado de exigencia perceptiva para el sistema autónomo dentro de los mapas seleccionados.

3.3.2.2 Generación de Trayectorias y Evaluación Métricas

Sobre los cuatro mapas base seleccionados (Town02, Town03, Town04 y Town10HD), se proyectó la generación de un corpus de validación compuesto por aproximadamente 40 trayectorias de referencia. La estructuración de estas rutas exigió una selección estratégica de los puntos de inicio (spawn points) y de finalización, distribuidos meticulosamente para forzar al agente autónomo a transitar por la máxima variedad de topologías presentes en cada entorno (rotondas, intersecciones asimétricas, incorporaciones y vías de alta capacidad).

Sin embargo, para dotar a esta selección de rigor analítico y evitar la clasificación subjetiva de los escenarios (por ejemplo, catalogar cualitativamente una ruta como "fácil" o "difícil"), surgió la necesidad de establecer una formulación matemática objetiva. Para ello, se diseñó e implementó una métrica de dificultad global (M_{total}), cuyo propósito es cuantificar la carga cognitiva, perceptiva y de control que una trayectoria específica exige al planificador del vehículo.

$$M_{total} = k_0 \cdot M_L + k_1 \cdot M_D + k_2 \cdot M_{seq} + k_3 \cdot M_{lp} \quad (3.1)$$

Donde los coeficientes empíricos k_n actúan como factores de ponderación que permiten calibrar la importancia relativa de cada componente dentro del simulador. Cada término de la ecuación modela una dimensión específica de la dificultad de navegación:



(a)



(b)

Figura 3.1: Vistas del entorno Town 02. (a) Vista aérea general del mapa. (b) Vista detallada de la calle principal (High Street).



(a)



(b)

Figura 3.2: Vistas del entorno Town 03. (a) Vista aérea general del mapa. (b) Vista detallada de la rotonda.



(a)



(b)

Figura 3.3: Vistas del entorno Town 04. (a) Vista aérea general del mapa. (b) Vista detallada del paso elevado (overpass).



(a)



(b)

Figura 3.4: Vistas del entorno Town 10. (a) Vista aérea general del mapa. (b) Vista detallada de la intersección (junction).

- Proporción Longitudinal (M_L): evalúa la desviación espacial del trayecto. Se calcula como la relación entre la distancia total de la ruta alterada por desvíos ($\bar{L}$) y la longitud de la ruta ideal original (L). No obstante, durante el modelado se determinó que el simple incremento de la distancia no es el factor más crítico, por lo que su peso es $k_0 = 1$.
- Carga de Decisiones (M_D): cuantifica la exigencia de maniobrabilidad. Este parámetro define la relación entre las decisiones que se toman en la ruta alterada por obstáculos ($\bar{D}$) entre las decisiones que se tomarían en la ruta ideal original (D). Para la carga de decisiones se decidió establecer el peso $k_1 = 0.7$, de forma que esté atenuado en comparación con las otras variables.
- Complejidad Secuencial (M_{seq}): este término resulta fundamental para capturar la saturación de decisiones. Las trayectorias que obligan al vehículo a tomar múltiples decisiones en intervalos de tiempo y espacio muy cortos se consideran operativamente más exigentes. Si la distancia d_i entre dos puntos de decisión consecutivos es menor que un umbral de seguridad mínimo ($d_{\min}$), se aplica una penalización local acumulativa $g(d_i)$, formulada mediante una función de decaimiento:

$$g(d_i) = \left(\frac{d_{\min} - d_i}{d_{\min}} \right)^p \quad (3.2)$$

De este modo, decisiones muy espaciadas no penalizan, mientras que intersecciones consecutivas muy próximas disparan el valor de M_{seq} . El peso establecido para este factor es de $k_2 = 1.3$.

- Penalización por Bucles (M_{lp}): aborda el grado de dificultad añadido en situaciones topológicas complejas donde los cortes viales fuerzan al agente a cruzar un mismo nodo (intersección o rotonda) en múltiples ocasiones. Para garantizar la validez metodológica, la generación de trayectorias se rige estrictamente por el principio de "rutas mínimas"; es decir, se descartan los bucles artificiales o carentes de justificación lógica (como rodear una rotonda indefinidamente), aplicando este factor únicamente cuando la oclusión del entorno obliga iterativamente a recalcular la salida a través de nodos ya transitados. Para modelar matemáticamente este fenómeno, la métrica calcula la dificultad acumulada iterando sobre todos los nodos de la ruta mediante la siguiente formulación:

$$M_{lp} = \sum_{k=1}^n I_k \quad (3.3)$$

Donde la dificultad individual aportada por cada intersección (I_k) se define como:

$$I_k = (v_k - 1) \cdot \left(\alpha \cdot \frac{L_k}{L_{ref}} \right) \quad (3.4)$$

Las variables involucradas en este modelo recogen tanto la frecuencia de paso como las proporciones geométricas del recorrido:

- n : representa el número total de cruces o intersecciones evaluadas a lo largo de la trayectoria.
- v_k : cuantifica el número de veces que el vehículo atraviesa la intersección
- α : define un coeficiente empírico de ponderación (valor arbitrario de calibración). Su función es amplificar o atenuar el peso del bucle en función de los requerimientos del experimento.
- L_k : expresa la distancia (en metros) que abarca geométricamente el cruce o bucle evaluado.
- L_{ref} : constituye la distancia total de la ruta original o ideal, sirviendo como factor de normalización para evaluar el impacto relativo del bucle sobre el recorrido íntegro.

El peso establecido para el factor M_{lp} (3.3) es $k_3 = 6.5$.

3.4 Subsistema de Navegación y Conducción Autónoma

3.4.1 Extracción Topológica y Planificación Global de Trayectorias

El desarrollo de un sistema de navegación autónoma requiere, como pilar fundamental, que el vehículo sea capaz de comprender el entorno estático en el que opera y de calcular secuencias de movimiento óptimas para desplazarse de un punto a otro. En este subapartado se describe el proceso de modelado del espacio transitable y la aplicación de algoritmos de búsqueda heurística para la generación de la ruta global.

3.4.1.1 Extracción de la topología en CARLA

El entorno simulado en CARLA fundamenta la definición de sus redes viales en el estándar OpenDRIVE, el cual proporciona una descripción detallada de la geometría de las carreteras, la disposición de los carriles y la estructura de las intersecciones. La extracción de esta base topológica se realiza a través de la API del simulador, empleando la clase especializada `GlobalRoutePlanner`. La inicialización de este módulo ejecuta una consulta al servidor para recuperar la topología del mapa como una colección de segmentos de carretera, donde cada segmento queda acotado por un par de objetos `waypoint` que representan su punto de entrada y salida.

En la arquitectura de CARLA, un `waypoint` trasciende el concepto de una simple coordenada espacial. Además de encapsular su posición en el espacio tridimensional mediante las componentes (x, y, z) , un `waypoint` integra información semántica vital para la navegación, incluyendo el identificador de la vía (`road_id`), la sección (`section_id`), el carril específico (`lane_id`) y una variable booleana que indica su pertenencia a una intersección. Para alcanzar un nivel de detalle apropiado para la posterior ejecución de la ruta, el sistema discretiza los segmentos de la carretera insertando `waypoints` intermedios. Esta discretización se rige por un parámetro de resolución de muestreo (`sampling_resolution`). En la implementación de este proyecto, gestionada por el módulo `Navigation.py`, dicha resolución se ha establecido empíricamente en 2.0 metros, un valor que garantiza una densidad de guiado suficiente sin incurrir en una penalización del rendimiento computacional.

A fin de profundizar en la gestión del espacio navegable, es imperativo destacar que, en la arquitectura vial de CARLA, las intersecciones o `junctions` se operan como entidades lógicas discretas (`carla.Junction`) que encapsulan explícitamente todos los pares de conectividad y trayectorias permitidas entre las vías que convergen en su interior. Esta modelización abstracta impone una limitación operativa crítica cuando el vehículo autónomo se desplaza por la mitad de un segmento vial convencional o tramo regular; al carecer estas secciones de carretera de opciones intrínsecas de bifurcación, resulta imposible asociar comandos semánticos de maniobra (tales como giros evasivos laterales) a dicho espacio lineal exacto. Por consiguiente, ante la detección de una obstrucción en un tramo vial de un solo sentido, el subsistema de replanificación se ve obligado a realizar una exploración geométrica anticipada a lo largo de la topología del carril actual para localizar el umbral de entrada de la próxima intersección válida. Esta dependencia topológica implica que, si el vehículo se encuentra excesivamente alejado de un cruce, o si se supera el radio de búsqueda máxima, la incapacidad de alterar localmente la estructura de aristas viales restringe la viabilidad de un desvío alternativo inmediato. El planificador global queda entonces supeditado a mantener la directriz por el carril actual, imposibilitado para trazar una ruta de escape hasta que una zona de decisión con flexibilidad en el grafo sea formalmente alcanzada o liberada por la topología del mapa.

Con el fin de ilustrar esta diferencia técnica, se desarrolló un script de depuración visual utilizando la API de CARLA (Repositorio [21]). En la Figura 3.5 se aprecia claramente cómo se gestiona el espacio de

la intersección: por un lado, su abstracción lógica define el cruce mediante pares de waypoints de entrada y salida sin describir la geometría intermedia; por otro lado, el `GlobalRoutePlanner` transforma esa lógica en una trayectoria física navegable, discretizando el camino según el parámetro de resolución (`sampling_resolution`).



Figura 3.5: Representación visual de la topología de una intersección en CARLA. Las líneas continuas (amarillo) ilustran las conexiones lógicas directas entre vías convergentes (pares de entrada-salida del objeto `carla.Junction`), mientras que la discretización puntual (azul) muestra la trayectoria interpolada generada por el `GlobalRoutePlanner`.

3.4.1.2 Construcción del grafo navegable

Una vez recopilados los datos topológicos brutos proporcionados por la API del simulador CARLA, se transforman en una estructura matemática discreta que soporta la evaluación algorítmica de trayectorias. Por tanto, la red de carreteras se ha modelado computacionalmente como un grafo dirigido $G = (V, E)$, empleando para su construcción y gestión en memoria la librería especializada `networkx` de Python.

En esta formulación abstracta, el conjunto de vértices V corresponde a los waypoints de entrada y salida identificados en la topología vial. Cada vértice $v \in V$ se representa mediante sus coordenadas físicas espaciales (x, y, z) y se asocia a un identificador numérico inequívoco. Por su parte, el conjunto de aristas dirigidas E modela las transiciones legales y físicamente transitables entre los vértices.

Para dotar al algoritmo de búsqueda de la información semántica y geométrica necesaria, a cada arista $e_{u,v} \in E$ (que conecta el nodo u con el nodo v) se le asigna un diccionario de atributos operacionales, entre los que destacan:

- Trayectoria discreta (`path`): una lista interna de waypoints intermedios que conforman la geometría tridimensional del segmento, separados por una resolución de muestreo predefinida.
- Vectores direccionales (`entry_vector`, `exit_vector`): vectores unitarios que describen la orientación cinemática del vehículo al ingresar y abandonar el segmento, fundamentales para la posterior evaluación geométrica de los giros e intersecciones.

- Clasificación topológica (*intersection*): un valor booleano que determina si la arista pertenece a un cruce o intersección.
- Coste de travesía (*length*): el peso matemático base de la arista, determinado inicialmente por la cantidad de waypoints que contiene el segmento. Este valor actúa como la heurística principal para la minimización de la función de coste durante la planificación.

El proceso de construcción del grafo garantiza su completitud geométrica mediante la ejecución de rutinas de saneamiento topológico. Específicamente, se implementa una lógica de resolución de tramos sin salida (*loose ends*), integrando al sistema aquellos segmentos viales desconectados, y se incorporan aristas lógicas transversales. Estas últimas representan la viabilidad espacial de ejecutar cambios de carril; al tratarse de desplazamientos laterales paralelos al flujo de conducción, se les asigna un peso algorítmico nulo ($L = 0$), permitiendo al planificador considerarlos sin penalización de distancia.

Como complemento visual a la formulación matemática descrita, la Figura 3.6b expone la extracción gráfica bidimensional (plano XY) del grafo de navegación $G = (V, E)$ generado en memoria para uno de los entornos de simulación. Esta abstracción topológica permite apreciar de forma directa los elementos fundamentales sobre los que opera el planificador de rutas.

Finalmente, la estructura de datos resultante, compuesta por el grafo dirigido de pesos ajustables y el registro espacial de vértices, constituye el marco operativo exacto que requiere el algoritmo de planificación geométrica. Es sobre este modelo donde el planificador global (gestionado por la clase `GlobalRoutePlanner`) ejecuta el algoritmo de búsqueda heurística A*.

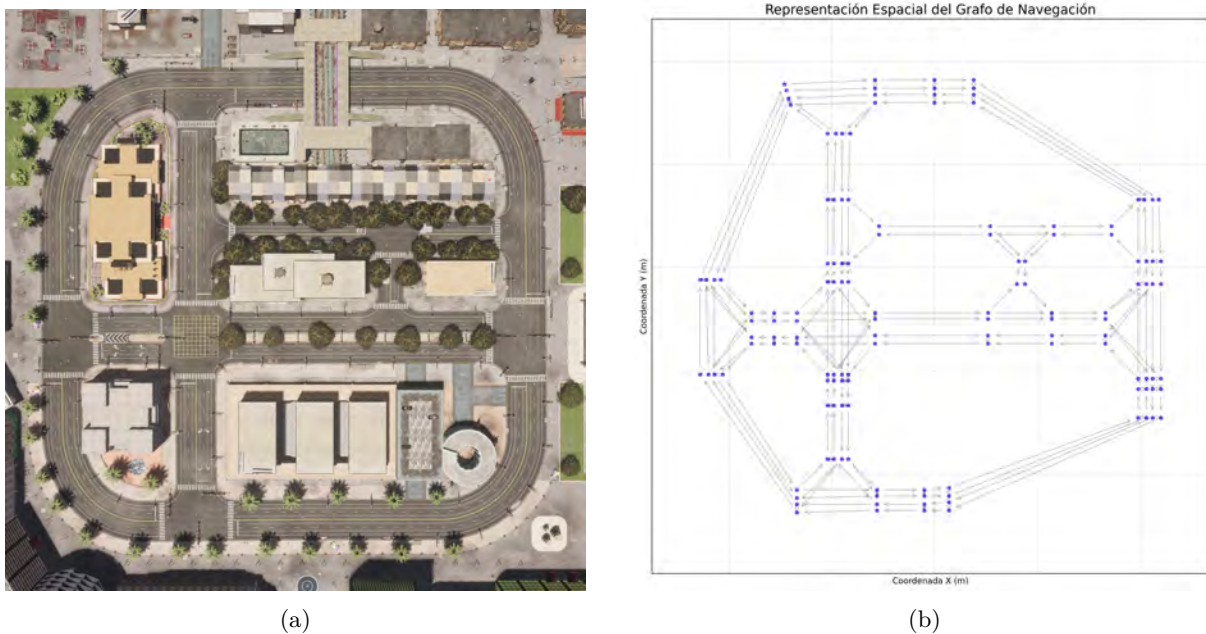


Figura 3.6: Comparativa topológica del entorno de simulación. (a) Vista aérea tridimensional del mapa físico en CARLA. (b) Representación espacial bidimensional del grafo de navegación dirigido $G = (V, E)$ extraído de dicha topología. Se aprecia la correspondencia directa entre los cruces físicos y la densidad de vértices y aristas en la abstracción matemática.

3.4.1.3 Planificación Geométrica (Global) mediante Algoritmo A*

Como comentamos en anteriores apartados, la planificación geométrica del trayecto se centraliza en la clase `GlobalRoutePlanner` proporcionada por la API del simulador CARLA. El objetivo de este

módulo es calcular el camino de mínimo coste entre un punto de origen y un destino, transformando la geometría continua del mundo simulado en un vector de instrucciones navegables de alto nivel. Para resolver este problema respetando las restricciones de tiempo real del subsistema de conducción autónoma, la arquitectura del planificador ejecuta un pipeline secuencial estructurado en tres fases algorítmicas fundamentales:

Fase 1: Abstracción Topológica y Construcción del Grafo

Previo a la evaluación matemática de rutas, el sistema debe discretizar el mapa en una estructura de datos transitable. Esta fase ha sido detallada en el apartado anterior.

Fase 2: Búsqueda del Camino Óptimo e Inyección Heurística

Una vez construido el grafo, el método `_localize` proyecta las coordenadas de origen y destino sobre sus aristas correspondientes. A continuación, el método `_path_search` ejecuta el algoritmo heurístico para calcular la ruta óptima.

El algoritmo A^* selecciona el siguiente nodo a expandir minimizando una función de coste total estimado, $f(n)$, definida como:

$$f(n) = g(n) + h(n) \quad (3.5)$$

- Coste Real Acumulado $g(n)$: representa el coste total desde el origen hasta el nodo n . Para que el vehículo pueda esquivar obstáculos en tiempo real basándose en la detección del subsistema de percepción, el espacio de costes del grafo se altera dinámicamente antes de la evaluación. La función de coste acumulado se define como:

$$g(n) = \sum_{i \in \text{path}(n_{start}, n)} c(e_i) \quad (3.6)$$

Donde $c(e_i)$ es el coste topológico final de cada arista que compone el camino. Este coste corresponde a la longitud física del tramo más una penalización aditiva P si la arista se encuentra dentro del radio de influencia de un obstáculo detectado:

$$c(e_i) = \begin{cases} \text{length}(e_i) + P & \text{si } e_i \text{ está afectada por un obstáculo} \\ \text{length}(e_i) & \text{en caso contrario} \end{cases} \quad (3.7)$$

Este procedimiento, detallado en el **Algoritmo 3.1**, incrementa artificialmente el peso topológico de las rutas bloqueadas, obligando al algoritmo a explorar trayectorias alternativas.

- Función Heurística $h(n)$: estima el coste restante desde el nodo n hasta el destino. Se implementa en el método `_distance_heuristic` calculando la distancia euclídea tridimensional:

$$h(n) = \sqrt{(x_{destino} - x_n)^2 + (y_{destino} - y_n)^2 + (z_{destino} - z_n)^2} \quad (3.8)$$

Dado que la distancia en línea recta no sobrestima el coste de navegación, la heurística resulta admisible, lo cual asegura matemáticamente el cálculo de la trayectoria óptima.

Data: Grafo $G = (V, E)$, Lista de obstáculos detectados OBS , Radio de influencia r , Penalización P

Result: Grafo G con pesos topológicos actualizados

begin

```

    Inicializar diccionario hash original_lengths
    for cada obstáculo  $O \in OBS$  do
        for cada arista  $e = (u, v) \in E$  do
             $W \leftarrow$  lista de waypoints discretizados en  $e$ 
             $afectada \leftarrow$  Falso
            for cada waypoint  $w \in W$  do
                 $d \leftarrow \sqrt{(w.x - O.x)^2 + (w.y - O.y)^2 + (w.z - O.z)^2}$ 
                if  $d < r$  then
                     $afectada \leftarrow$  Verdadero
                    break // Optimización: basta con un waypoint afectado
            if  $afectada = Verdadero$  then
                if  $e \notin original\_lengths$  then
                     $original\_lengths[e] \leftarrow e.length$ 
                 $e.length \leftarrow original\_lengths[e] + P$  // Penalización heurística
    return  $G$ 

```

Algoritmo 3.1: Alteración Dinámica del Espacio de Costes

Fase 3: Extracción Semántica y Álgebra Vectorial (**trace_route**)

Una vez calculado el camino óptimo, el algoritmo devuelve una lista secuencial de nodos lógicos. El paso final lo ejecuta el método `trace_route`, que transforma este modelo topológico en una trayectoria navegable para el sistema de control.

Este método recorre los segmentos de la ruta para interpolar una serie detallada de objetos `carla.Waypoint`. Al mismo tiempo, asigna a cada tramo una instrucción de maniobra mediante la enumeración *RoadOption* (por ejemplo, *Straight*, *Left*, *Right*). Para determinar la dirección de los giros en las intersecciones, el método `_turn_decision` utiliza operaciones de álgebra vectorial.

Para determinar la dirección del giro, el método `_turn_decision` extrae del grafo topológico los vectores unitarios precalculados de entrada ($\vec{v}_{in}$) y salida ($\vec{v}_{out}$) de la intersección. La maniobra se clasifica calculando el producto vectorial de ambos:

$$\vec{w} = \vec{v}_{in} \times \vec{v}_{out} \quad (3.9)$$

Como la navegación se realiza en el plano horizontal (x, y) , basta con evaluar el signo de la componente ortogonal (w_z) . De acuerdo con la regla de la mano derecha, este valor define unívocamente hacia dónde se realiza el giro como se muestra en la Figura 3.7.

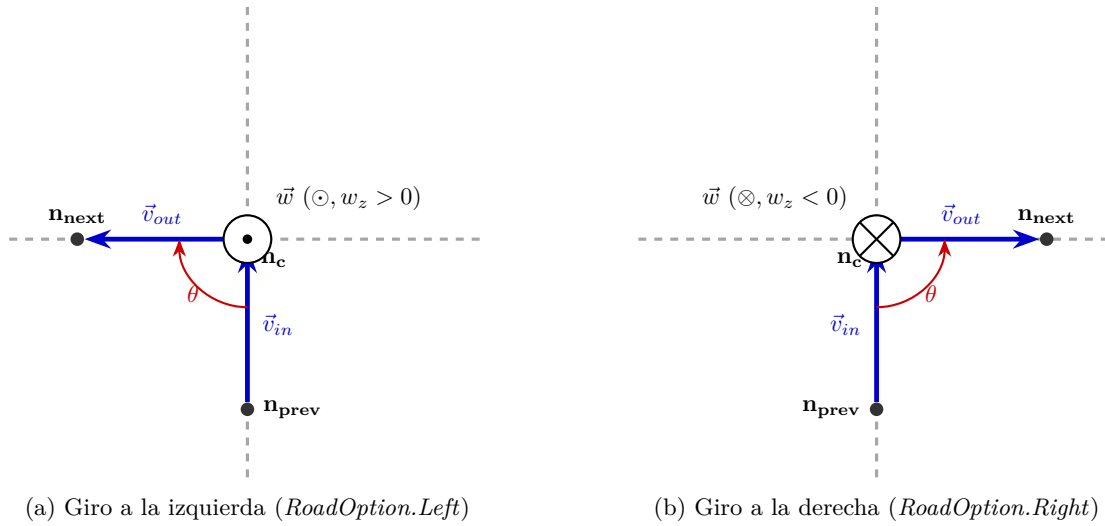


Figura 3.7: Asignación de maniobras (*RoadOption*) en CARLA. El producto vectorial de las trayectorias de entrada ($\vec{v}_{in}$) y salida ($\vec{v}_{out}$) define el giro: (a) giro a la izquierda para $w_z > 0$, y (b) giro a la derecha para $w_z < 0$.

Como resultado de estas tres fases, se obtiene una secuencia de tuplas ($waypoint_i, RoadOption_i$). Esta estructura define la ruta global que el subsistema de navegación procesará para controlar los actuadores longitudinales y laterales del vehículo.

3.4.2 Módulo de Control y Ejecución

El Módulo de Control y Ejecución constituye la última etapa en la cadena de procesamiento del subsistema de navegación autónoma. Su responsabilidad fundamental radica en la traslación de las trayectorias geométricas calculadas y los imperativos de seguridad en comandos físicos ejecutables por los actuadores del vehículo virtual (acelerador, freno y ángulo de dirección). En el entorno de simulación CARLA, este proceso se ha modelado mediante una arquitectura de control jerárquico instanciada en la clase *BehaviorAgent* definida por CARLA, la cual hereda las capacidades de seguimiento de trayectoria de un agente base (*BasicAgent*) y le superpone una capa de comportamiento dinámico y reactivo.

La ejecución del control se invoca cíclicamente dentro del bucle principal del sistema a través del método `run_step()`. Este método opera como una máquina de estados determinista que evalúa el entorno inmediato del vehículo y prioriza la toma de decisiones basada en la criticidad de los eventos. La jerarquía de evaluación comienza por el cumplimiento estricto de las señales de tráfico, seguido por la evasión de peatones y vehículos colindantes y, finalmente, la adaptación de la velocidad en intersecciones y tramos rectos.

Una vez que la capa de comportamiento ha definido tanto el waypoint objetivo a seguir como la velocidad longitudinal deseada, la responsabilidad de la traducción física recae sobre el planificador local. Es en este subcomponente donde las discrepancias espaciales y cinemáticas se transforman en variables de control continuas. Para ello, el sistema confía en lazos de control realimentados, típicamente fundamentados en controladores Proporcional-Integral-Derivativo (PID). El control longitudinal se encarga de gobernar la aceleración y la frenada del vehículo.

El controlador minimiza el error entre la velocidad actual del vehículo, obtenida de la telemetría del simulador, y la velocidad de referencia estipulada por el agente de comportamiento. La ley de control genérica aplicada toma la siguiente forma:

$$u_{long}(t) = K_{p,long}e_v(t) + K_{i,long} \int e_v(t)dt + K_{d,long} \frac{de_v(t)}{dt} \quad (3.10)$$

donde $e_v(t)$ es el error de velocidad en el instante t , y los parámetros K corresponden a las ganancias de las acciones proporcional, integral y derivativa. Una salida positiva del controlador u_{long} se satura y se mapea directamente al comando de aceleración (throttle), mientras que un valor negativo se interpreta y traduce en la acción de frenado (brake).

Paralelamente, el control lateral es responsable de mantener al vehículo centrado en el carril. Este proceso calcula el error de desviación lateral (cross-track error) así como el error de orientación (heading error) con respecto a la tangente del waypoint geométrico más cercano en la ruta planificada. Mediante una ley de control análoga a la longitudinal, el controlador lateral genera la señal de control $u_{lat}(t)$, la cual se traduce en el grado de rotación del volante.

Finalmente, la síntesis de estos cálculos se encapsula en una estructura de datos nativa del simulador denominada `carla.VehicleControl`. Esta estructura normaliza los parámetros físicos requeridos por el motor de físicas del vehículo en rangos definidos:

- Aceleración: $[0.0, 1.0]$
- Freno: $[0.0, 1.0]$
- Dirección: $[-1.0, 1.0]$, donde -1.0 representa el giro máximo a la izquierda y 1.0 a la derecha.

En situaciones excepcionales de alto riesgo dictaminadas por la lógica de colisión, el método `emergency_stop()` sobrescribe las directrices del planificador local instanciando un control de emergencia que anula por completo la aceleración e impone la máxima fuerza de frenado.

Para dotar al sistema de navegación de una flexibilidad adaptativa frente a diferentes escenarios operacionales, el módulo de control incorpora una capa de toma de decisiones parametrizada mediante la abstracción de perfiles de comportamiento. Esta arquitectura permite desacoplar el control cinemático de bajo nivel de los criterios heurísticos de seguridad y eficiencia de alto nivel. En la implementación realizada, la diferenciación conductual se articula a través de tres clases predefinidas heredadas del módulo de navegación de CARLA: Cautious, Normal y Aggressive.

Cada uno de estos perfiles actúa como un contenedor de variables de estado y umbrales cinemáticos que modifican directamente la sensibilidad de los gestores de colisión, evasión de peatones y seguimiento de vehículos. Los parámetros fundamentales que definen y diferencian el comportamiento del agente se describen a continuación:

- Velocidad Máxima (`max_speed`): establece el límite absoluto de velocidad longitudinal que el vehículo autónomo puede adoptar bajo condiciones de flujo libre, independientemente de que las condiciones geométricas de la vía permitan un límite superior.
- Margen del Límite de Velocidad (`speed_lim_dist`): determina la diferencia operativa con respecto al límite legal de velocidad de la vía establecido por los mapas del simulador. Mientras que un perfil agresivo reduce este margen al mínimo para aproximarse estrictamente al límite de la vía, un perfil prudente impone un margen holgado de seguridad sub-límite.
- Tiempo de Seguridad (`safety_time`): representa el umbral crítico empleado en la evaluación del Tiempo Hasta la Colisión (*TTC*). Este parámetro modifica directamente la antelación temporal con la que el vehículo inicia las maniobras de desaceleración progresiva al detectar un obstáculo frontal.

- Distancia de Frenado de Emergencia (`braking_distance`): define el umbral crítico de seguridad en metros. Si la distancia a un obstáculo es menor a este valor, el sistema interrumpe el control habitual y activa la detención inmediata mediante el método `emergency_stop()`.
- Umbral de Proximidad Mínima (`min_proximity_threshold`): define la distancia máxima a la que el vehículo detecta y evalúa a otros actores en la vía. Este valor es clave para ajustar la sensibilidad del sistema en maniobras como el seguimiento a corta distancia (*tailgating*).
- Tasa de Decremento de Velocidad (`speed_decrease`): determina cuánto se reduce la velocidad objetivo cuando el sistema detecta un vehículo circulando delante. Este ajuste permite que el coche decelere de forma suave y mantenga la distancia de seguridad.

La arquitectura modular del código facilita enormemente la creación de nuevos perfiles de conducción. Para añadir una política personalizada —como un modo de eficiencia energética o un comportamiento más defensivo— no es necesario modificar los controladores PID ni el planificador local. Basta con ajustar los parámetros del objeto `self._behavior` durante la inicialización del `BehaviorAgent`. Así, cambiar el carácter del vehículo se simplifica a la configuración de sus umbrales de seguridad.

Como etapa final del ciclo de control, el script principal de integración invoca la directiva `vehicle.apply_control(control)`, inyectando el vector de estado resultante en la API de CARLA. Esta instrucción actualiza inmediatamente las fuerzas físicas y la cinemática del modelo de dinámica de vehículo de Unreal Engine en el siguiente tick de la simulación, cerrando así el lazo entre el razonamiento de inteligencia artificial y la interacción mecánica en el mundo virtual.

3.5 Percepción Multimodal y Replanificación Dinámica

La percepción del entorno es uno de los pilares fundamentales en la conducción autónoma. En este proyecto, la aproximación clásica basada puramente en la segmentación semántica o en redes neuronales convolucionales (CNN) de detección de objetos ha sido sustituida por el uso de un Modelo de Visión-Lenguaje (VLM, por sus siglas en inglés). Concretamente, se ha integrado el modelo multimodal Llama 4 Scout, el cual permite no solo detectar obstáculos, sino razonar sobre el contexto de la escena vial y emitir respuestas estructuradas para el subsistema de toma de decisiones del vehículo.

3.5.1 Características de Llama 4 Scout en el Contexto de la Conducción Autónoma

El modelo seleccionado para la integración es `meta-llama/llama-4-scout-17b-16e-instruct` [22]. Este modelo destaca por su arquitectura multimodal de 17 billones de parámetros, optimizada para seguir instrucciones precisas (*instruct-tuned*) y procesar entradas que combinan texto e imágenes simultáneamente. Su estructura interna se compone de tres bloques funcionales principales ilustrados en la Figura 3.9:

- Codificador de Visión: para procesar la información geométrica y cromática de las cámaras del vehículo, Llama 4 Scout integra un codificador visual basado típicamente en la arquitectura Vision Transformer (ViT) [23]. El ViT segmenta cada una de las imágenes de entrada en una matriz de parches discretos de tamaño fijo. Cada parche se aplanan y se proyecta linealmente para generar una secuencia de vectores de características visuales que capturan las dependencias espaciales de la escena mediante mecanismos de *self-attention*.

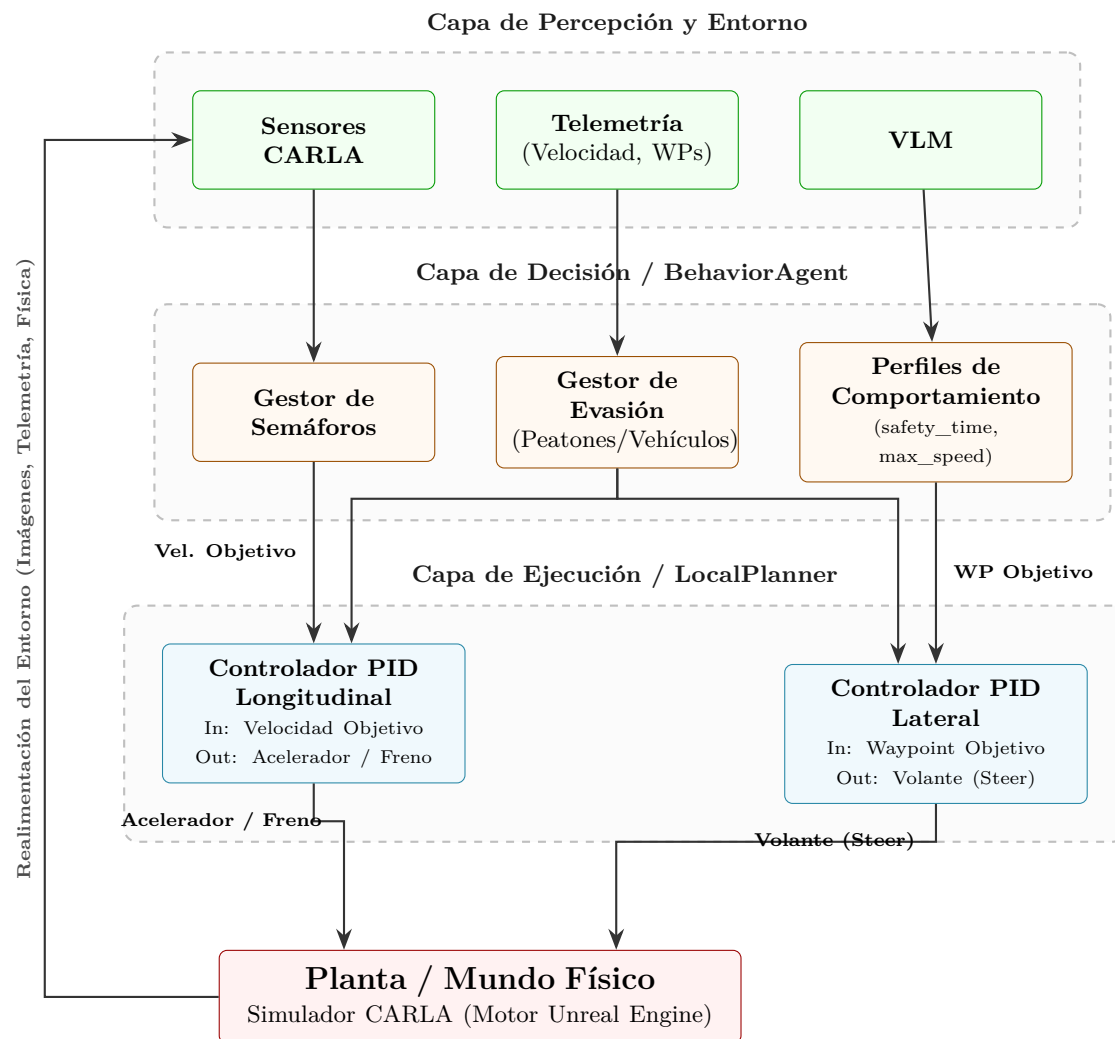


Figura 3.8: Arquitectura de control jerárquico del subsistema de navegación autónoma y lazo de interacción con el entorno de simulación CARLA.

- **Módulo de Proyección Lineal o Adaptador de Modalidades:** dado que las representaciones vectoriales del codificador de visión y del modelo de lenguaje nativo residen en espacios dimensionales nativos diferentes, la arquitectura implementa una red de proyección [24]. Este componente actúa como un puente, transformando los tokens visuales de salida del ViT a la misma dimensión del espacio de embeddings del modelo de lenguaje. Gracias a este alineamiento, el modelo puede tratar las características de las imágenes exactamente igual que si fuesen tokens de texto generados a partir de palabras.
- **Backbone del Modelo Autoregresivo:** el núcleo central del sistema está constituido por el decodificador Transformer de Llama 4, el cual implementa una arquitectura MoE (Mixture of Experts) [25]. En lugar de ser un bloque denso tradicional, el modelo posee una capacidad total de aproximadamente 109 billones de parámetros distribuidos en 16 expertos independientes (16e). Durante la inferencia, un mecanismo de enrutamiento dinámico evalúa cada token y activa únicamente el subconjunto de expertos óptimo para esa tarea, poniendo en marcha una escala de 17 billones de parámetros activos por token (17b).

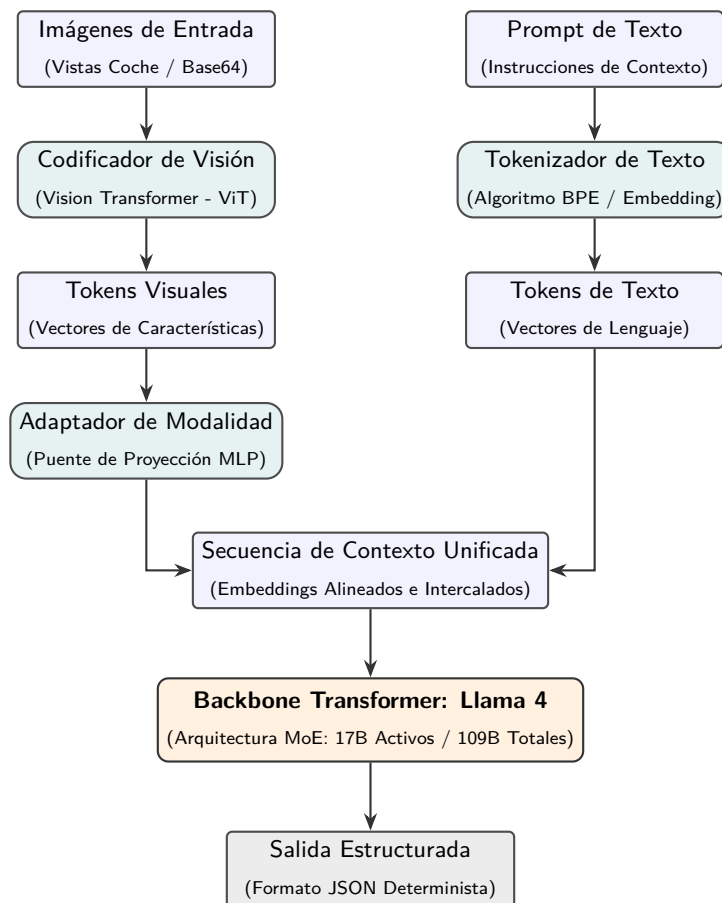


Figura 3.9: Diagrama de bloques de la arquitectura de sistemas de Llama 4 Scout. Se ilustra la ingesta multimodal en paralelo y la convergencia de datos en el mecanismo de Mezcla de Expertos (MoE).

Pese a encontrarnos en un entorno de simulación como CARLA, se ha tenido en cuenta la latencia, por lo que la inferencia no se ejecuta en local, sino a través de la API RESTful de Groq. Groq utiliza Unidades de Procesamiento de Lenguaje (LPUs), un hardware de inferencia diseñado específicamente para maximizar el rendimiento y minimizar la latencia temporal en modelos generativos. Esto permite que el vehículo consulte el estado del entorno de forma iterativa sin paralizar excesivamente el hilo principal

de la simulación. Además, el modelo cuenta con la capacidad nativa de forzar la salida en un formato estructurado mediante el parámetro `response_format: {"type": "json_object"}`, lo cual es estrictamente necesario para que el código en Python (el controlador del vehículo) pueda parsear de forma determinista variables booleanas (bloqueo), flotantes (distancia, nivel de confianza) y listas (direcciones libres).

3.5.2 Proceso de Iteración y Evolución Arquitectónica

El desarrollo e integración del módulo de percepción visual no constituyó un diseño estático, sino que fue el resultado de un proceso empírico de iteración a través de varias versiones arquitectónicas implementadas en el entorno CARLA. Esta evolución metodológica fue estrictamente necesaria para mitigar cuellos de botella computacionales (latencia de inferencia), reducir la tasa de falsos positivos y solucionar la pérdida de contexto espacial periférico en escenarios urbanos complejos.

Fase Inicial: Redundancia Sensorial y Problemas de Campo de Visión (FOV)

En las fases iniciales del desarrollo, la arquitectura del sistema se basó en la fusión y redundancia de sensores para garantizar la seguridad activa del agente. Para ello, el vehículo ego se equipó con una cámara RGB estándar y un sensor de segmentación semántica.

- Implementación del sistema Override: la cámara de segmentación actuaba como un mecanismo de seguridad determinista de bajo nivel. Se programó un algoritmo para analizar el canal de etiquetas (canal de color Red en el buffer de CARLA) en una región de interés (ROI) que abarcaba del 50% al 90% del alto de la imagen y del 30% al 70% del ancho. Si la presencia de etiquetas catalogadas como críticas —identificadas internamente con los valores [4, 10, 20, 21], correspondientes a peatones y vehículos— superaba un umbral de bloqueo del 5% (`BLOCKAGE_THRESHOLD = 0.05`), el sistema forzaba una detención de emergencia sin esperar la deliberación del Modelo de Visión-Lenguaje (VLM).
- Evolución del Control y Escenarios: durante esta fase, el control puramente reactivo fue sustituido por el gestor de rutas dinámicas `RouteManager` acoplado a un `BehaviorAgent` para permitir desviaciones físicas del vehículo ante obstáculos (`FIXED_OBSTACLE_DISTANCE = 100.0` metros proyectados). Adicionalmente, se integró un analizador de archivos YAML (`obstacles.yaml`, `inicio_fin.yaml`) para la instanciación procedimental de escenarios y props físicos (obstáculos) en la simulación.
- Limitación visión de túnel detectada: para reducir el coste computacional y enfocar la atención del modelo, se aplicaba un recorte central a la imagen RGB de entrada (extrayendo únicamente desde el 20% al 80% de las dimensiones de alto y ancho). Si bien esto optimizaba el enfoque frontal, este recorte central generaba un punto ciego severo, provocando una "visión de túnel" que impedía al agente anticipar obstáculos laterales en maniobras críticas como la salida de intersecciones.

Fase de Transición: Autonomía del Modelo Multimodal

Este conjunto de iteraciones marcaron un punto de inflexión arquitectónico al eliminar por completo la dependencia del sensor de segmentación semántica y su lógica de override asociada.

En entornos de conducción autónoma del mundo real, disponer de un mapa de segmentación semántica perfecto (ground truth) sin un modelo pesado de red neuronal paralela es inviable. Para evaluar el verdadero potencial cognitivo de Llama 4 Scout como sistema End-to-End de percepción, este debía operar de forma aislada a partir de los píxeles crudos.

Mediante técnicas de Prompt Engineering, la IA asumió la carga cognitiva completa. Demostró una notable capacidad heurística para distinguir espacialmente entre objetos. Mediante la inyección de reglas estrictas en su contexto (System Prompt), el modelo aprendió a diferenciar entre mobiliario urbano inofensivo situado en las aceras (fuera de la calzada) y objetos críticos con su base de apoyo físicamente ubicada sobre el asfalto (conos de tráfico, barreras y escombros).

Fase Final: Arquitectura Multi-Vista y Trazabilidad Heurística

Para solventar de forma definitiva la pérdida de contexto periférico sin distorsionar la imagen (lo cual degradaría la densidad de información que recibe el ViT del modelo Llama), se descartó el recorte central unificado en favor de un enfoque de percepción espacial distribuida, denominado Arquitectura Multi-Vista.

Segmentación Espacial de la Imagen: el frame original (capturado con un Field of View de 95 grados) se divide matricialmente en tres sub-imágenes superpuestas como ilustra la Figura 3.10, que garantizan la cobertura total del entorno frontal y lateral sin pérdida de objetos situados en las costuras de las imágenes:

- Vista Izquierda: recorte horizontal entre el 0% y el 40% del ancho del frame original.
- Vista Central: recorte horizontal entre el 30% y el 70%.
- Vista Derecha: recorte horizontal entre el 60% y el 100%.

Integración Multimodal Avanzada: estas tres imágenes se redimensionan a matrices de 400x400 píxeles, se codifican en Base64 y se envían en paralelo al modelo VLM. Llama 4 Scout evalúa la intersección en su conjunto, determinando qué vía de escape específica (izquierda o derecha) se encuentra libre para replanificar la trayectoria.

Optimización del Agente y Trazabilidad: en la versión definitiva, el perfil del controlador cinemático (BehaviorAgent) fue elevado al estado agresivo para evaluar la latencia del sistema bajo velocidades de aproximación mayores. Para garantizar la transparencia y explicabilidad de las decisiones del VLM, se implementó en el Head-Up Display (HUD) del simulador un panel de telemetría de historial, como se muestra en la Figura 3.11. Utilizando una cola circular (`collections.deque` de tamaño máximo 5), el sistema almacena y renderiza las últimas iteraciones de razonamiento del modelo. Para evitar la saturación visual de la interfaz, se programó un filtro que descarta las notas de la IA que sean idénticas a la última registrada en el historial, proporcionando así una herramienta de depuración limpia y eficiente en tiempo real. Adicionalmente, el radio heurístico de memoria para evitar bucles infinitos de replanificación en el mismo cruce se aumentó a 30 metros de distancia cartesiana entre el vehículo y el obstáculo registrado.

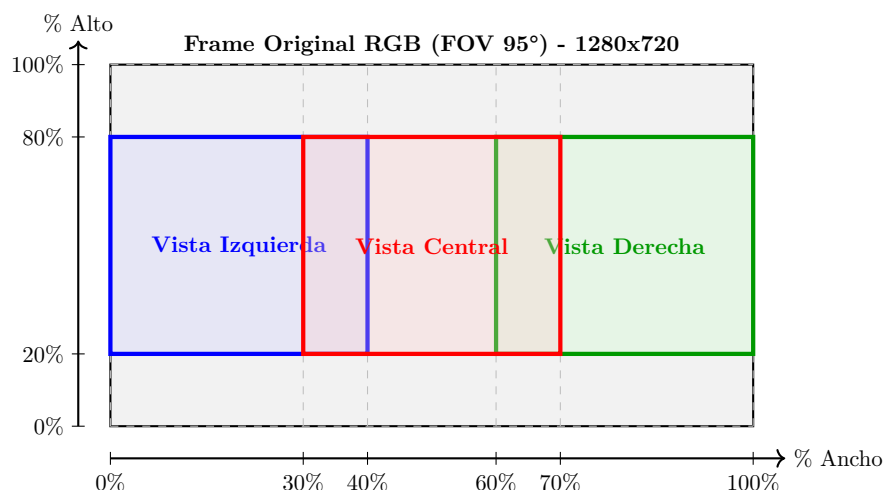


Figura 3.10: Esquema geométrico del preprocesamiento de la imagen (*Region of Interest*). En el eje vertical, se aplica un recorte entre el 20% y el 80% para descartar el cielo y el posible chasis del vehículo o asfalto inmediato. En el eje horizontal, las tres vistas mantienen un solapamiento del 10% para garantizar la conservación del contexto espacial en las transiciones visuales.



Figura 3.11: Interfaz de visualización (HUD) en CARLA. En el panel superior izquierdo se monitoriza la telemetría del vehículo junto con los parámetros del modelo multimodal. Los paneles derecho e inferior exponen el historial y la salida actual del razonamiento semántico de la IA.

3.5.3 Arquitectura de Comunicación y Pipeline de Procesamiento

La integración entre el entorno tridimensional de CARLA y el modelo remoto Llama 4 Scout se ha implementado mediante un diseño de hilos de ejecución concurrentes (multithreading). Esta decisión arquitectónica es imperativa para desacoplar las físicas del vehículo (que requieren alta frecuencia de actualización) de la inferencia de la IA (que opera a frecuencias menores). De este modo, se evitan bloqueos temporales (stutters) en el hilo principal del simulador. El flujo de datos sigue un pipeline secuencial estructurado en cuatro fases fundamentales.

3.5.3.1 Captura Sensorial y Preprocesamiento Multi-Vista

La primera fase del pipeline consiste en la adquisición de datos del entorno simulado y su adecuación geométrica y de formato antes de ser transmitidos al modelo multimodal. El vehículo ego está equipado con un sensor de visión principal generado mediante el sistema de *spawn* de componentes de CARLA (`sensor.camera.rgb`) configurado para emular una cámara frontal de amplio espectro. Este sensor se ubica estratégicamente mediante una transformación espacial con respecto al centroide del vehículo (específicamente a 2.8 metros en el eje longitudinal X , 2.3 metros en el eje vertical Z y con una inclinación angular o pitch de -5 grados), proporcionando una perspectiva óptima de la calzada y las intersecciones. La cámara captura imágenes a una resolución nativa de 1280x720 píxeles con un campo de visión (FOV) de 95 grados.

La adquisición de estos frames se rige por el modo síncrono del simulador CARLA (mediante ejecuciones iterativas de `world.tick()`), garantizando el determinismo de la simulación. Como se ilustra en la Figura 3.13, el Hilo Principal (CARLA) se encarga de extraer la telemetría y recibir la secuencia de imágenes a través de un mecanismo de *callback* basado en una cola de mensajes asíncrona (`queue.Queue()`). Esta estructura de datos thread-safe permite desacoplar la frecuencia de renderizado del simulador de los tiempos de inferencia del modelo subyacente. Los datos crudos (en formato buffer unidimensional) son transformados matricialmente mediante la librería NumPy a un tensor tridimensional de dimensiones $720 \times 1280 \times 4$, extrayéndose posteriormente los tres canales de color (RGB) relevantes para la percepción visual.

Tras capturar el fotograma en el hilo secundario de percepción (`vlm_worker`), el sistema ejecuta un preprocesamiento geométrico. Tal y como se ha detallado exhaustivamente en apartados anteriores, este proceso consiste en aplicar la técnica de recorte multivista, generando tres perspectivas parcialmente superpuestas. El objetivo de esta delimitación espacial es descartar la información visual irrelevante, garantizando que la inferencia del modelo se concentre de manera exclusiva en el área transitable, la señalización y los bordes de la calzada.

Finalmente, para mitigar el impacto de la latencia de red durante la transmisión del payload a la API de Groq (observado en el diagrama de secuencia de la Figura 3.13), las tres matrices resultantes sufren un proceso de compresión y codificación. Cada subimagen es redimensionada mediante interpolación a una resolución de 400×400 píxeles. Posteriormente, se codifican empleando el estándar JPEG con un factor de compresión del 80%, y se serializan a una cadena de texto en formato Base64 (utf-8). Este empaquetado optimizado resulta crítico, dado que las imágenes deben ser encapsuladas dentro de un esquema JSON (payload multi-imagen) junto con las instrucciones estructuradas (prompt) dirigidas al LLM multimodal, permitiendo así una transferencia HTTP POST eficiente sin comprometer excesivamente la integridad de las características visuales del entorno.

3.5.3.2 Inferencia Semántica y Sincronización de Estado

Una vez preprocesada la información visual, el sistema requiere traducir los datos crudos a un nivel de abstracción semántica que el planificador pueda interpretar. Esta tarea recae sobre el modelo Llama-4 Scout, accesible a través de la API de Groq, cuya inferencia se ejecuta de manera asíncrona para no penalizar la tasa de fotogramas por segundo (FPS) del simulador CARLA.

El proceso de inferencia semántica se articula mediante la técnica de Prompt Engineering. El payload transmitido en la petición HTTP 3.13 no solo contiene las imágenes en Base64, sino también un conjunto de instrucciones perimetrales que definen el comportamiento del modelo como un sistema experto de percepción para conducción autónoma. Para garantizar la fiabilidad del análisis, se imponen reglas lógicas

estrictas al modelo: se le instruye explícitamente para discernir entre obstáculos críticos ubicados sobre la vía transitable (asfalto, carriles, intersecciones) y elementos que forman parte del mobiliario urbano o zonas seguras (aceras, aparcamientos adyacentes, vallas perimetrales). Esta discriminación geométrica y semántica es vital para evitar falsos positivos que inducirían frenadas fantasma o replanificaciones innecesarias.

Para garantizar la interoperabilidad entre el modelo de lenguaje natural y el código de control del vehículo, se fuerza al modelo a visualmente estructurar su salida estrictamente en formato JSON. La respuesta del servidor es interceptada, decodificada y parseada para instanciar una estructura de datos inmutable y tipada mediante la clase `dataclass` de Python, denominada `PerceptionEvent`. Esta estructura actúa como el contrato de información entre el módulo de percepción y el de control temporal, encapsulando las siguientes variables de estado:

- `blocked`: variable booleana que indica de forma binaria si la trayectoria actual o las salidas de la intersección presentan un obstáculo infranqueable.
- `confidence_blocked`: métrica de tipo flotante que cuantifica el grado de certidumbre del modelo sobre la predicción espacial.
- `obstacle_relative_position`: cadena de texto que mapea espacialmente la posición del obstáculo en función de los recortes visuales analizados (`left`, `front`, `right` o `none`).
- `free_directions`: lista de direcciones alternativas viables deducidas por el modelo a partir de la escena actual.
- `notes`: cadena de texto libre donde el modelo explicita el razonamiento lógico subyacente a su decisión (por ejemplo, identificando conos de obra en la salida derecha).
- `source`: identificador del origen del evento (VLM Multi-View frente a inyecciones manuales).

Mantener la coherencia de los datos en este entorno concurrente requiere una sincronización de estado precisa. Como se detalla en la Figura 3.13, el hilo secundario (`vlm_worker`) opera en un bucle continuo regulado por *flags* como `exit_flag` y `mutex` implícitos en las escrituras atómicas de Python. El ciclo de vida de este hilo y sus transiciones se rigen mediante la Máquina de Estados Finitos detallada en la Figura 3.12. Durante el tiempo que dura la transacción HTTP (estado `WAITING_API`), el hilo activa la variable de estado `ai_is_busy`, lo que permite al hilo principal reajustar dinámicamente la física del simulador (aplicando el `SLOW_MOTION_FACTOR`) y alertar visualmente en el HUD sobre la latencia actual (`last_ai_latency`), ofreciendo una ruta de escape hacia el estado `IDLE` en caso de producirse un *timeout* o excepción de red.

Una vez que el modelo retorna el JSON, se transita al estado `UPDATING_STATE` donde la variable global `last_event` se sobrescribe de manera segura. Además, para proporcionar una trazabilidad en tiempo real y evitar el almacenamiento de estados redundantes, se implementa una cola de doble extremo (deque) de tamaño fijo denominada `ai_history`. Este historial almacena secuencialmente el razonamiento textual del modelo (`notes`), filtrando deliberadamente las respuestas duplicadas consecutivas. Esta memoria semántica a corto plazo facilita la depuración y demuestra la evolución del razonamiento de la IA a medida que el vehículo se aproxima al obstáculo.

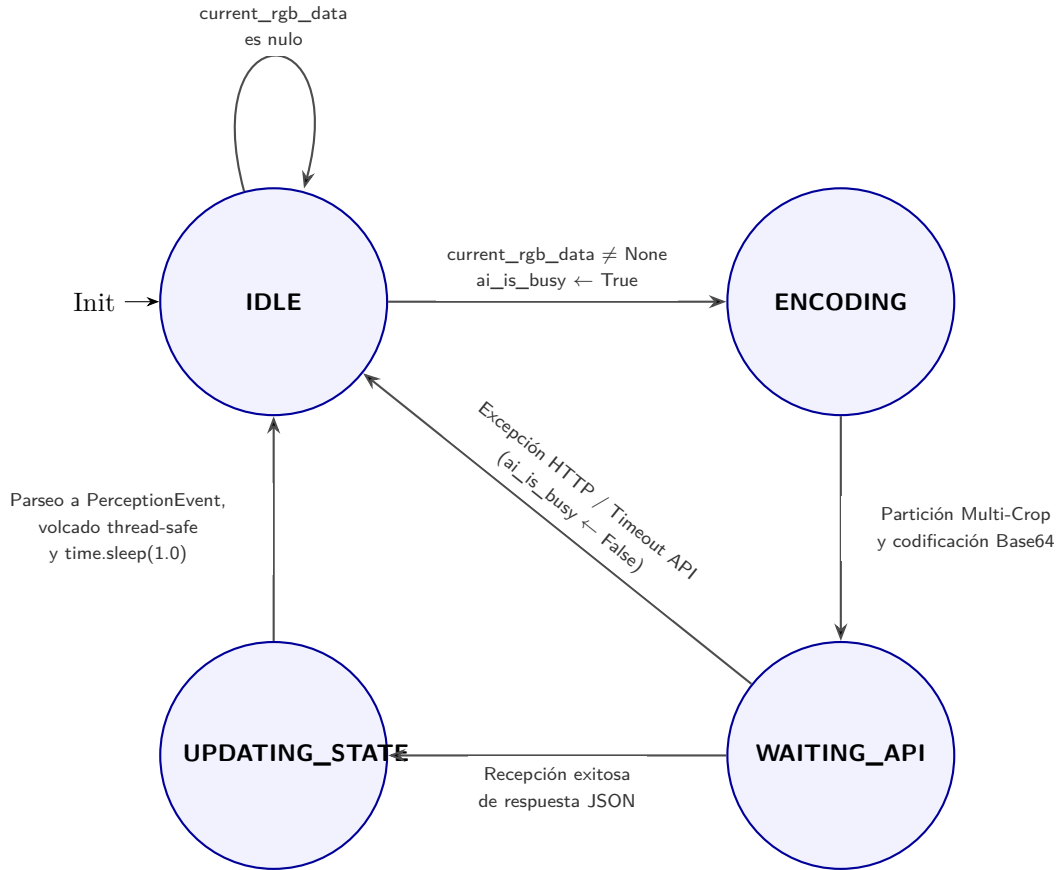


Figura 3.12: Diagrama de Autómata de Estados Finitos (FSM) que rige el ciclo de vida concurrente del hilo de percepción `vlm_worker`.

3.5.3.3 Replanificación Dinámica Online

Para adaptarse a un entorno urbano cambiante, el vehículo, además de la detección de obstáculos, debe ser capaz de modificar su trayectoria global en tiempo de ejecución. Este proceso, denominado replanificación dinámica online, constituye el eje de la toma de decisiones del sistema y actúa como nexo entre la comprensión semántica proporcionada por el VLM y el módulo de navegación topológica de CARLA.

El mecanismo de replanificación se evalúa en cada iteración del bucle principal (gobernado por `world.tick()`). El sistema monitorea continuamente el estado de la variable `active_event`, la cual encapsula la última predicción del hilo de percepción o, en su defecto, las inyecciones de eventos manuales introducidas mediante interrupciones de teclado para fines de depuración y validación (teclas I, J, L). Cuando el evento activo certifica la presencia de un bloqueo en la ruta (`active_event.blocked == True`), se desencadena el protocolo de evasión.

Para garantizar la estabilidad del sistema y evitar bucles computacionales infinitos o comportamientos oscilatorios (donde el planificador recalcula incesantemente la misma ruta ante el mismo obstáculo persistente en el campo de visión), se ha implementado un mecanismo de histéresis espacial o "memoria topológica", cuyo flujo de ejecución se detalla en la primera fase del Algoritmo 3.2. El sistema mantiene un registro dinámico (`handled_obstacle_locations`) que almacena las coordenadas tridimensionales absolutas del vehículo en el instante en que se procesó una replanificación. Antes de invocar al módulo de cálculo de rutas, el algoritmo verifica la distancia euclídea entre la posición actual del ego-vehículo (`ego_loc`) y el histórico de coordenadas registradas. Si el vehículo se encuentra en un radio inferior a 30 metros de una ubicación previamente gestionada, el sistema asume que el obstáculo detectado pertenece

a la misma intersección o segmento vial, clasificando el evento como redundante (`is_new_event = False`) e ignorando la petición de desvío. Esta zona de exclusión espacial permite al vehículo maniobrar y superar el cruce sin interferencias de nuevas inferencias sobre el mismo bloqueo.

Si la validación espacial confirma que se trata de un evento nuevo, se delega el recálculo al gestor de rutas mediante el método `route_manager.check_and_replan(ego_trans, active_event)`. Este componente cruza los datos geométricos del grafo de navegación de CARLA con la información semántica proporcionada por el modelo (la posición relativa del obstáculo y las direcciones libres sugeridas). Si el algoritmo subyacente encuentra una ruta alternativa viable (`replanned == True`), el nuevo conjunto de waypoints se inyecta directamente en el controlador de comportamiento (`BehaviorAgent`) mediante la función `set_global_plan(new_route)`. En caso contrario, se registra la imposibilidad de evasión, pero la posición se almacena igualmente en el registro de exclusión para evitar la saturación del hilo principal.

Adicionalmente, la transición fluida entre la ruta original y la alternativa requiere una gestión precisa del seguimiento de waypoints. Para ello, se ha desarrollado la función de optimización `get_current_wp_index`, sistematizada en la segunda fase del Algoritmo 3.2. En lugar de iterar sobre la totalidad de los nodos de la ruta (lo cual resulta computacionalmente ineficiente e introduce el riesgo de que el vehículo "salte" erróneamente a tramos paralelos de la misma vía), esta función restringe el espacio de búsqueda. El algoritmo calcula la distancia mínima únicamente proyectando una ventana de *look-ahead* limitada a los próximos 30 waypoints sucesivos. Esto asegura una convergencia rápida y fiable del índice actual, permitiendo al sistema actualizar correctamente la telemetría del HUD (`wp_current / wp_total`) y suministrar al bucle de control local las referencias de estado adecuadas para la actuación final.

Data: Posición actual ego_loc , Evento IA active_event , Registro handled_locations , Ruta actual R , Índice actual idx

Result: Ruta global R y nuevo índice local mejor_idx

begin

```

// Fase 1: Evaluación de Histéresis y Replanificación
if  $\text{active\_event.blocked} = \text{Verdadero}$  then
     $\text{nuevo\_evento} \leftarrow \text{Verdadero}$ 
    for  $\text{cada } \text{loc} \in \text{handled\_locations}$  do
        if  $\text{DistanciaEuclídea}(\text{ego\_loc}, \text{loc}) < 30.0$  then
             $\text{nuevo\_evento} \leftarrow \text{Falso}$ 
            break
    if  $\text{nuevo\_evento} = \text{Verdadero}$  then
         $\text{replan}, \text{nueva\_ruta} \leftarrow \text{ReplanificarRuta}(\text{ego\_loc}, \text{active\_event})$ 
         $\text{handled\_locations.Añadir}(\text{ego\_loc})$  // Evitar bloqueos futuros
        if  $\text{replan} = \text{Verdadero}$  then
             $R \leftarrow \text{nueva\_ruta}$ 
             $\text{idx} \leftarrow 0$  // Reinicio sobre la nueva trayectoria

// Fase 2: Búsqueda Local de Waypoint
 $\text{min\_dist} \leftarrow \infty$ 
 $\text{mejor\_idx} \leftarrow \text{idx}$ 
 $\text{limite} \leftarrow \min(|R|, \text{idx} + 30)$  // Acotación computacional de la ventana
for  $i \leftarrow \text{idx}$  to  $\text{limite} - 1$  do
     $\text{dist} \leftarrow \text{DistanciaEuclídea}(\text{ego\_loc}, R[i].\text{transform.location})$ 
    if  $\text{dist} < \text{min\_dist}$  then
         $\text{min\_dist} \leftarrow \text{dist}$ 
         $\text{mejor\_idx} \leftarrow i$ 
return  $R, \text{mejor\_idx}$ 

```

Algoritmo 3.2: Mecanismo de Histéresis Espacial y Búsqueda de Waypoint (Look-ahead)

3.5.3.4 Actuación y Bucle de Control Local

La etapa final del pipeline de procesamiento cierra el lazo de control transformando las directrices topológicas de alto nivel, previamente validadas o replanificadas, en comandos mecánicos de bajo nivel. Esta fase es gobernada por el bucle principal de ejecución y se apoya en el controlador de comportamiento propio de la API de CARLA (BehaviorAgent).

Para la experimentación, el agente local se inicializa bajo un perfil de comportamiento, por ejemplo, agresivo ($\text{behavior} = \text{'aggressive'}$). Esta parametrización permite al vehículo ejecutar maniobras de evasión y giros en intersecciones con mayor agilidad, un requisito deseable cuando se evalúa la capacidad de respuesta inmediata tras una replanificación inducida por el modelo multimodal.

En cada iteración temporal del simulador, el sistema evalúa la condición de parada mediante el método agent.done() , el cual verifica si el vehículo ha alcanzado con éxito las coordenadas espaciales del destino final. En caso negativo, se invoca a la función central del controlador local, agent.run_step() . Este método computa internamente las variables de estado cinemático del ego-vehículo respecto al waypoint objetivo más inmediato, calculando las señales de actuación requeridas. El resultado es un

objeto de la clase `carla.VehicleControl`, que encapsula valores normalizados en el rango $[0, 1]$ para el acelerador (`throttle`) y el freno (`brake`), así como un valor en el rango $[-1, 1]$ para el ángulo de dirección (`steer`). Estas señales analógicas se transmiten directamente al vehículo mediante la instrucción `vehicle.apply_control(control)`.

De forma paralela a la actuación física, el bucle de control se encarga de la monitorización y representación visual de la telemetría en tiempo real. La velocidad escalar del vehículo se extrae a partir de su vector de velocidad tridimensional, aplicando la norma euclídea y su correspondiente factor de conversión a kilómetros por hora:

$$v = \sqrt{v_x^2 + v_y^2 + v_z^2} \times 3.6 \quad (3.11)$$

Esta métrica, junto con el estado de inferencia de la IA, la latencia de red, la posición actual en la ruta y el historial de razonamiento almacenado en la memoria semántica, se renderizan sobre el frame original utilizando primitivas gráficas de la biblioteca OpenCV. Esta interfaz de visualización, comentada en apartados anteriores, proporciona un mecanismo de validación empírica fundamental durante los ensayos de simulación.

Finalmente, el determinismo y la estabilidad temporal del lazo de control se garantizan mediante una gestión estricta del tiempo de ciclo, cuya lógica de ejecución se describe en el Algoritmo 3.3. El tiempo objetivo por fotograma (`target_frame_time`) se ajusta dinámicamente en función de la carga de inferencia, aplicando un factor de ralentización (`SLOW_MOTION_FACTOR`) si el hilo secundario se encuentra ocupado (`ai_is_busy`). Al concluir el cómputo de cada iteración, se calcula el tiempo remanente:

$$t_{sleep} = t_{target} - (t_{actual} - t_{inicio_{bucle}}) \quad (3.12)$$

Si $t_{sleep} > 0$, el hilo principal se suspende explícitamente durante ese intervalo, asegurando que la frecuencia de actuación se mantenga fiel a los parámetros físicos establecidos (1.0/20.0 segundos). Toda esta arquitectura de control se encapsula en una estructura de manejo de excepciones (`try-finally`) que, ante la finalización de la ruta o una interrupción del usuario, garantiza una finalización ordenada de los subprocesos (`ai_thread.join()`), la destrucción de los actores físicos en Unreal Engine (cámara y vehículo) para prevenir fugas de memoria, y la restauración de la configuración asíncrona original del servidor CARLA.

3.6 Ajustes y Optimización del Modelo Multimodal

Habiendo descrito detalladamente en los apartados anteriores la arquitectura y el funcionamiento interno del modelo Llama 4 Scout (incluyendo su codificador de visión, el adaptador de modalidades y su backbone autoregresivo MoE), es necesario abordar los desafíos de su implementación práctica en el sistema de conducción.

Aunque el modelo fundacional destaca por su extraordinaria capacidad de comprensión semántica, el comportamiento base de un Modelo de Visión-Lenguaje (VLM) no está diseñado intrínsecamente para satisfacer las estrictas exigencias de determinismo, conciencia espacial y baja latencia que requiere la conducción autónoma. Su integración directa en el entorno dinámico del simulador CARLA requirió, por tanto, un trabajo iterativo de calibración empírica y ajuste.

Data: Frecuencia base FPS_{normal} , Factor ralentización F_{slow} , Estado VLM `ai_is_busy`

Result: Señales de actuación al controlador y estabilización de la simulación

begin

```

 $\Delta t_{fijo} \leftarrow 1.0 / FPS_{normal}$ 
while Verdadero (Bucle Principal) do
   $t_{start} \leftarrow \text{ObtenerTiempoActual}()$ 
  if ai_is_busy = Verdadero then
     $t_{target} \leftarrow \Delta t_{fijo} \times F_{slow}$  // Expansión temporal para estabilización
  else
     $t_{target} \leftarrow \Delta t_{fijo}$ 
  Mundo.Tick() // Avanzar motor físico de Unreal Engine
   $control \leftarrow \text{Agente.EjecutarPaso}()$  // Cálculo PID longitudinal y lateral
  Vehículo.AplicarControl( $control$ )
  RenderizarHUD( $t_{target}$ )
   $t_{elapsed} \leftarrow \text{ObtenerTiempoActual}() - t_{start}$ 
   $t_{sleep} \leftarrow t_{target} - t_{elapsed}$  // Cálculo de remanente temporal
  if  $t_{sleep} > 0$  then
    SuspenderHilo( $t_{sleep}$ ) // Mantiene el determinismo del sistema

```

Algoritmo 3.3: Bucle de Control Local y Sincronización Temporal Dinámica

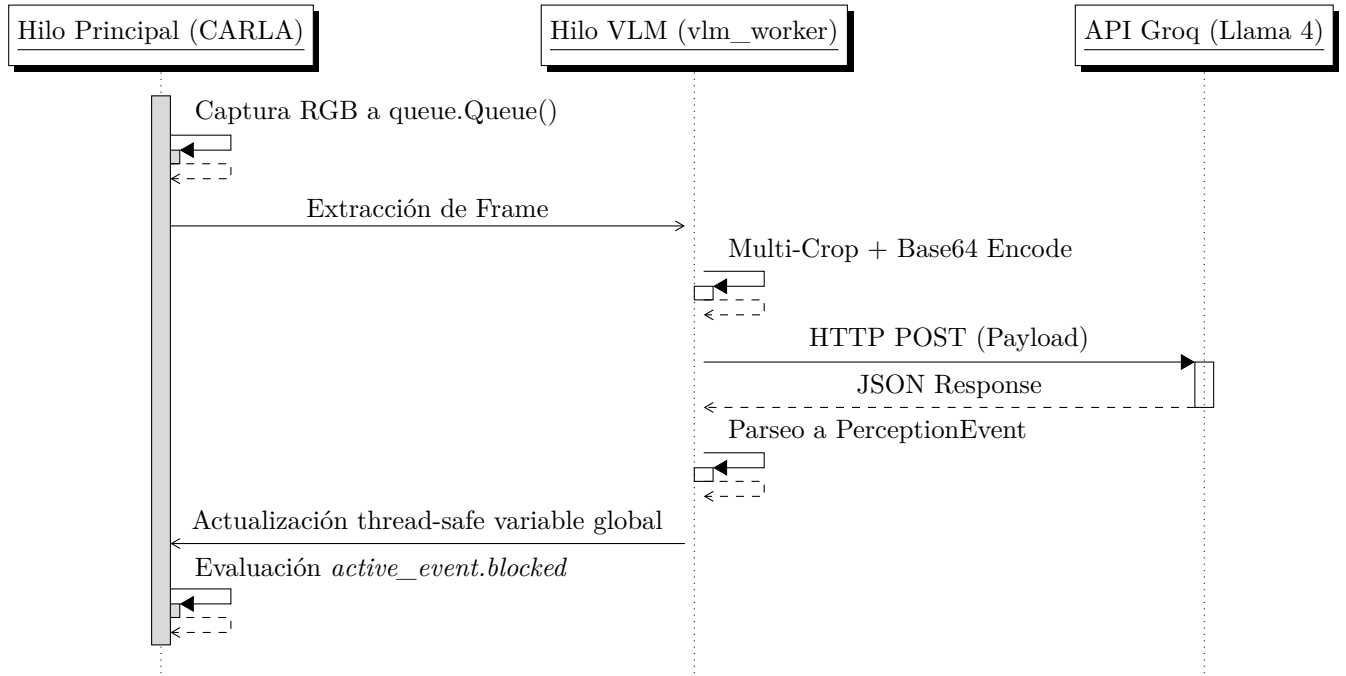


Figura 3.13: Diagrama de secuencia del pipeline concurrente. El hilo principal mantiene la ejecución de físicas (`world.tick`) sin bloqueos mientras el hilo secundario gestiona la latencia de red con la API remota.

En los siguientes subapartados se explican los mecanismos y estrategias específicas que se han implementado para optimizar el rendimiento y la fiabilidad de la red neuronal. Concretamente, se detallará cómo se ha parametrizado el proceso de inferencia de la IA, la evolución arquitectónica hacia un sistema de percepción multiobjetivo (*multi-view*) para recuperar el contexto lateral, y la aplicación de técnicas de *Prompt Engineering* como *Chain-of-Thought* y *Negative Prompting*, para corregir las limitaciones de razonamiento espacial del modelo en entornos urbanos complejos.

3.6.0.1 Parametrización de la Inferencia

Para garantizar que el modelo multimodal funcionara como un módulo de clasificación lógico y no como un asistente conversacional creativo, se ajustaron drásticamente sus hiperparámetros de generación.

En primer lugar, la temperatura del modelo se estableció en un valor muy bajo (0.1). Al acercar este parámetro a cero, se minimiza la aleatoriedad en la predicción del siguiente token, forzando a la red a producir respuestas altamente deterministas y consistentes ante el mismo *frame* de entrada. En segundo lugar, se restringió la longitud de salida a un máximo de 800 tokens y se forzó una estructura de respuesta nativa mediante el formato JSON (`response_format: {"type": "json_object"}`). Esto asegura que el planificador geométrico pueda deserializar los datos de percepción (como el estado de bloqueo, la posición relativa o las direcciones libres) en tiempo real, sin requerir un procesamiento de lenguaje natural (NLP) intermedio y computacionalmente costoso.

3.6.0.2 Evolución del Contexto Visual y Coste Computacional

La aproximación inicial del sistema consistía en procesar una única vista frontal. Sin embargo, esta configuración presentaba limitaciones significativas de "efecto túnel" en los cruces urbanos como ya comentamos en los apartados anteriores, perdiendo información sobre las vías adyacentes. Para mejorar la anticipación lateral, se implementó un sistema de inferencia multi-imagen. En esta configuración, el modelo recibe simultáneamente tres recortes visuales solapados (Izquierda, Centro y Derecha) extraídos de una cámara de gran angular, los cuales se etiquetan de forma explícita en el *prompt*.

Aunque esta mejora incrementó la eficacia al detectar obstáculos laterales, introdujo un compromiso técnico (trade-off): un aumento sustancial en el consumo de tokens por llamada y, por tanto, una mayor latencia. Para compensar esta carga de procesamiento, fue necesario optimizar aún más las instrucciones del sistema para no sacrificar los *FPS* durante la simulación.

3.6.0.3 Estrategias de *Prompt Engineering*

Durante las pruebas experimentales, se detectó que el modelo tenía dificultades con el razonamiento espacial en 3D. Aunque la red clasificaba los objetos correctamente (vallas, bancos, papeleras o muros), le faltaba contexto espacial. El modelo tendía a establecer asociaciones directas ("Valla = Peligro = Bloqueo"), sin importar si el objeto estaba a varios metros, en la acera o en un aparcamiento. Como los elementos fuera de la calzada no impiden la navegación, este exceso de sensibilidad causaba falsos positivos y replanificaciones innecesarias.

Para resolver esta sobre-sensibilidad, se rediseñó el contexto de entrada (*system prompt*) integrando dos técnicas consolidadas en el campo de la Inteligencia Artificial Generativa [26]:

- *Negative Prompting*: se introdujeron reglas restrictivas explícitas para acotar el espacio de búsqueda del modelo [26]. Se instruyó a la red para ignorar categóricamente objetos situados en aceras, detrás

de vallas, en zonas de aparcamiento aisladas o cualquier mobiliario urbano adyacente a la vía. Al definir explícitamente lo que no es un obstáculo, se logró reducir drásticamente el ruido del entorno.

- *Chain-of-Thought*: se obligó al modelo a realizar un análisis procedimental (*step-by-step*) del entorno antes de dar una respuesta [27]. En lugar de evaluar directamente el estado de bloqueo, el *prompt* fuerza a la IA a analizar topológicamente dónde está apoyado el objeto físico en relación con los límites reales de la calzada (como bordillos o líneas de carril).

Gracias a estas modificaciones, el modelo evalúa de forma fiable la viabilidad del espacio transitable (*drivable path*), lo que proporciona una mayor robustez al planificador frente a los escenarios urbanos de CARLA.

3.7 Conclusiones

En conclusión, este tercer capítulo establece la base técnica y metodológica del sistema de conducción autónoma propuesto. Se ha logrado pasar de un diseño modular a la implementación de un sistema de control que une percepción semántica y planificación. Separar las tareas en componentes especializados, en lugar de utilizar un modelo único (monolítico), ha resultado ser una estrategia eficaz para crear un sistema escalable y fácil de interpretar.

Por una parte, la adaptación del simulador CARLA y la selección de escenarios ofrecen un entorno de pruebas sólido. Además, la creación de una métrica matemática de dificultad global (M_{total}) supone un aporte metodológico clave. Esta métrica permite medir con precisión la carga cognitiva, perceptiva y geométrica del sistema, superando así las limitaciones de las evaluaciones puramente cualitativas. Sobre esta base, representar la red vial mediante grafos dirigidos ha permitido implementar un algoritmo de planificación A* dinámico. Este algoritmo ha demostrado su eficacia al actualizar los costes del mapa en tiempo real, penalizando las zonas con obstáculos para forzar la creación de rutas alternativas.

Por otra parte, el avance técnico más destacado de este capítulo es la integración del modelo multimodal Llama 4 Scout como sistema de percepción, sustituyendo a la segmentación clásica. El desarrollo de un preprocesamiento multi-vista ha solucionado el problema inicial del "efecto túnel", asegurando así el contexto lateral en intersecciones críticas. Además, el diseño de una arquitectura concurrente (multithreading) y el uso de la API de Groq han sido fundamentales para separar la alta frecuencia de la simulación

A nivel de optimización del razonamiento semántico, la aplicación de técnicas de Prompt Engineering ha sido clave. La combinación de directrices de *Negative Prompting* y *Chain-of-Thought* ha corregido las dificultades del modelo en su razonamiento espacial tridimensional. Gracias a estos ajustes, el sistema ahora evalúa el entorno de forma fiable: sabe diferenciar correctamente el mobiliario urbano situado fuera de la calzada de los obstáculos reales, lo que reduce en gran medida las replanificaciones de ruta innecesarias.

En conjunto, la integración de estos subsistemas (la inferencia semántica asíncrona, la replanificación topológica con histéresis para evitar bucles y el control cinemático mediante PID) ofrece una solución técnica sólida al problema planteado. Una vez establecidas las bases algorítmicas del simulador y configurado el modelo multimodal, el trabajo da paso a la fase de experimentación. El siguiente capítulo analizará los resultados obtenidos al ejecutar las trayectorias de referencia, evaluando formalmente la precisión para evadir obstáculos y la fiabilidad de las decisiones del sistema en escenarios de distinta complejidad.

Capítulo 4

Resultados

4.1 Introducción

En este capítulo se exponen los resultados de la validación experimental del planificador de trayectorias desarrollado. El objetivo principal de esta fase es evaluar el rendimiento algorítmico y la viabilidad técnica del sistema al integrar un modelo multimodal (Llama4 Scout) como núcleo de percepción y toma de decisiones para la navegación en entornos urbanos simulados mediante CARLA.

Para garantizar una exposición estructurada de los datos, el documento se articula en base a los componentes del proceso de evaluación. Inicialmente, se detalla la configuración del entorno de simulación y las herramientas de telemetría implementadas para la monitorización en tiempo real. Seguidamente, se definen las métricas de evaluación empleadas para cuantificar tanto la precisión del módulo de visión como la eficacia de las rutinas de control. A continuación, se documenta la evolución iterativa de la arquitectura de percepción, justificando las modificaciones geométricas y de procesamiento aplicadas para corregir las deficiencias tempranas del modelo. Finalmente, se detalla el diseño del catálogo de casos de prueba utilizado y se presenta el análisis técnico del comportamiento del vehículo frente a restricciones topológicas, oclusiones visuales y escenarios de alta densidad semántica.

4.2 Entorno de Simulación y Condiciones de Prueba

Para evaluar el rendimiento del planificador de trayectorias y el módulo de percepción visual, se ha establecido un entorno de validación determinista empleando el simulador CARLA. La selección de mapas, la alteración topológica de las vías y el desarrollo de un sistema de instrumentación específico permiten cuantificar la respuesta del vehículo autónomo frente a obstáculos imprevistos.

4.2.1 Mapas y Topología Vial (CARLA)

Las pruebas experimentales se distribuyen en cuatro mapas con características estructurales y semánticas diferenciadas, como se comentó en la Sección 3.3.2.1 garantizando la evaluación del sistema en condiciones geométricas heterogéneas:

- Town02: establece la línea base de rendimiento en la detección de bloqueos frontales y transversales a baja velocidad.

- Town03: evalúa la robustez perceptual ante alteraciones del horizonte visual y restricciones circulares.
- Town04: evalúa la capacidad del planificador para gestionar bifurcaciones de alta velocidad y ejecutar desvíos macro-topológicos ante la pérdida de enlaces interurbanos.
- Town10HD: evalúa la tasa de falsos positivos del modelo multimodal frente a texturas superpuestas y ruido visual de fondo.

4.2.2 Mapas y Topología Vial (CARLA)

Para forzar los recálculos de ruta, se optó por una aproximación dinámica mediante la API de Python de CARLA, evitando así la alteración definitiva de los niveles base del simulador. El proceso consiste en la instanciación paramétrica (*spawn*) de actores estáticos sobre el área transitable en tiempo de ejecución. La gestión de estos escenarios se centraliza a través de un archivo de configuración YAML, y los bloqueos implementados abarcan:

- Estructuras continuas: vallas de obra y barreras de hormigón que bloquean de forma absoluta las vías principales.
- Balizamiento disperso: agrupaciones de conos de tráfico diseñados para estrechar los carriles o generar bloqueos integrados con el mobiliario urbano.
- Entornos de emergencia: vehículos de emergencias de gran tamaño (coches de policía, camiones de bomberos y ambulancias) combinados con señales luminosas y cajas de carga.
- Obstrucciones irregulares: montículos de tierra, rocas y escombros situados estratégicamente en pendientes o salidas de intersecciones para forzar bloqueos parciales.

4.2.3 Telemetría y Monitorización (HUD)

Para auditar el estado interno del algoritmo de navegación y los tiempos de inferencia del VLM en tiempo real, se ha desarrollado un Head-Up Display (HUD), como ya comentamos en apartados anteriores, y queda ilustrado en la Figura 3.11; superpuesto a la señal de vídeo principal del vehículo. Este panel de telemetría extrae e imprime los siguientes parámetros de ejecución:

- Dinámica del Vehículo: velocidad actual (km/h) y tasa de actualización del simulador (FPS).
- Métricas del Modelo Multimodal (VLM): estado de ejecución, latencia de inferencia por llamada, confirmación de detección de bloqueo (booleano True/False), origen de la detección (Manual/VLM) y estimación de la posición relativa del obstáculo (Frontal, Izquierda, Derecha).
- Gestión de Ruta: ratio de waypoints completados sobre el total planificado. El denominador de esta métrica se incrementa dinámicamente cuando el enrutador ejecuta una replanificación e inserta nuevos nodos en el grafo de búsqueda.
- Volcado de Razonamiento: un cuadro de texto en el área inferior de la interfaz que renderiza las notas generadas por el VLM en la inferencia actual. Esto permite correlacionar de manera visual el diagnóstico semántico de la red neuronal con el entorno proyectado en ese instante exacto.

4.3 Definición de las Métricas de Evaluación

Para cuantificar el desempeño conjunto del sistema de percepción visual y el algoritmo de planificación geométrica, se establece un conjunto de métricas divididas en tres dominios: la precisión de la inferencia neuronal, la eficiencia del control de navegación y la complejidad intrínseca del escenario.

4.3.1 Métricas de Percepción y Razonamiento (VLM)

El rendimiento del modelo multimodal (Llama4 Scout) se evalúa en función de su capacidad para segmentar semánticamente el área transitable y procesar el contexto espacial, así como su viabilidad para operar en el bucle de control del vehículo. En este sentido, una de las métricas principales es la Tasa de Falsos Positivos, que cuantifica las instancias en las que el modelo clasifica erróneamente un elemento del entorno como un bloqueo activo de la calzada. Este error desencadena una penalización injustificada en los pesos de los nodos correspondientes del grafo de búsqueda. Al suceder esto, el planificador se ve forzado a descartar trayectorias óptimas viables, lo que desemboca en desvíos innecesarios o, en mapas con conectividad restrictiva, en la imposibilidad algorítmica de alcanzar el destino. Por lo general, estos errores derivan de la pérdida de profundidad espacial en la imagen bidimensional, como ocurre al identificar un muro perimetral o mobiliario urbano ubicado en la acera como un elemento invasivo.

Por otra parte, la Tasa de Falsos Negativos registra las omisiones de obstáculos reales presentes en la trayectoria transitable. Esta métrica representa un fallo crítico de seguridad, ya que priva al algoritmo geométrico de la información previa necesaria para actualizar el grafo. En consecuencia, el controlador de bajo nivel (BehaviorAgent) se ve obligado a depender exclusivamente de la maniobra reactiva de emergencia, lo que incrementa significativamente la probabilidad de colisión. Estas omisiones suelen originarse por la mimetización cromática del obstáculo con el entorno o por alteraciones en la geometría proyectiva, tales como los cambios de inclinación (pitch) al circular por pendientes.

Finalmente, como dato adicional, observamos la Latencia de Inferencia (t_{inf}), definida como el tiempo en milisegundos que requiere el modelo visual para procesar el conjunto de imágenes y generar la estructura de respuesta en formato JSON. Esta métrica temporal no tiene la misma relevancia que las anteriores para el presente Trabajo, ya que no es el objetivo de esta experimentación alcanzar valores de latencia viables para un proyecto de ingeniería de este planificador. En cualquier caso, la latencia del razonamiento es el factor que acota el margen de anticipación espacial del vehículo para iniciar una deceleración cinemática segura.

4.3.2 Métricas de Planificación y Control

La eficacia operativa del RouteManager y el BehaviorAgent para resolver el recorrido bajo condiciones de incertidumbre se mide mediante dos indicadores deterministas. El primero de ellos es el Porcentaje de Ruta Completado, que establece la relación porcentual entre la distancia efectivamente navegada por el vehículo y la distancia total planificada hasta el objetivo. Un valor inferior al 100% indica un fallo terminal en la simulación, provocado generalmente por una colisión derivada de un falso negativo visual o por la incapacidad del algoritmo para resolver escenarios de restricción geométrica severa, como la navegación sostenida en corredores estrechos o balizados.

El segundo indicador corresponde a las Replanificaciones Activadas frente a Esperadas, el cual compara el número de recálculos de ruta ejecutados dinámicamente por el sistema contra la cantidad teórica de intervenciones topológicas inyectadas de forma controlada en el escenario. Un desajuste positivo, donde las activadas superan a las esperadas, evidencia la presencia de falsos positivos visuales o una inestabilidad

oscilatoria en la clasificación, fenómeno conocido como parpadeo semántico en largas distancias. Por el contrario, un desajuste negativo indica fallos persistentes de detección que impiden al planificador reaccionar ante los cambios del entorno.

4.3.3 Índice de Dificultad del Escenario (M_{total})

Para establecer un marco de validación comparativo, se define el escalar M_{total} , el cual parametriza la exigencia global de cada caso de prueba. Esta métrica queda descrita en la Sección 3.3.2.2.

En base al valor numérico, los escenarios se clasifican en tres umbrales de severidad, identificados mediante un código de color:

- Dificultad Baja ($M_{total} < 5.0$): color verde.
- Dificultad Media ($5.0 \leq M_{total} < 10.0$): color amarillo.
- Dificultad Alta ($M_{total} \geq 10.0$): color rojo.

4.4 Diseño y Justificación del Catálogo de Pruebas

La evaluación experimental se estructura mediante un catálogo de casos de prueba aislados e independientes. Este diseño metodológico responde a la necesidad de garantizar la reproducibilidad empírica y evitar el enmascaramiento de fallos algorítmicos críticos, un fenómeno común cuando se agregan resultados en métricas estadísticas globales. En el ámbito de la conducción autónoma, una tasa general de éxito elevada puede encubrir deficiencias sistemáticas en escenarios específicos. Por ello, la granularidad de este catálogo permite correlacionar de manera directa el estímulo visual procesado por el modelo multimodal con la respuesta cinemática y topológica del planificador geométrico. Con el fin de estandarizar este proceso, cada entrada del catálogo se organiza en torno a una estructura estricta dividida en cuatro bloques de información.

El primer bloque, relativo a la Identificación y Resultado, define el entorno base mediante el mapa de CARLA, el resultado práctico del trayecto (ya sea la llegada al destino o el tipo de colisión registrado) y el índice de dificultad escalar (M_{total}), lo que ofrece una referencia rápida sobre la viabilidad de la ruta. A continuación, el bloque de Condiciones del Escenario detalla las características de la obstrucción introducida en el entorno virtual. En este apartado se especifica la geometría de la vía afectada, la composición semántica del bloqueo (como barreras estructurales, vehículos de emergencia o balizamiento) y el objetivo técnico de la prueba, que puede consistir en forzar recálculos macrotopológicos o evaluar la sensibilidad periférica del modelo.

El tercer bloque corresponde a las Métricas de Evaluación, el cual se compone de una matriz cuantitativa orientada a auditar los dos módulos principales del sistema. Por un lado, mide la precisión espacial del VLM a través de la tasa de falsos positivos y negativos y, por otro, la eficiencia del algoritmo de control (RouteManager y BehaviorAgent), expresada mediante el porcentaje de ruta completada y la relación entre las replanificaciones esperadas y ejecutadas. Finalmente, el apartado de Observaciones y Análisis Técnico examina de manera detallada el comportamiento integral del sistema más allá del resultado binario de éxito o fracaso. Este último bloque documenta errores perceptuales vinculados a la resolución espacial y el comportamiento táctico del vehículo autónomo ante configuraciones de vía complejas o degradaciones del entorno.

A través de esta estructura documental, el análisis permite separar las limitaciones propias de la arquitectura de inferencia neuronal de las deficiencias algorítmicas del planificador, facilitando así un diagnóstico modular del sistema en su conjunto.

El catálogo completo está disponible para consultar en el Repositorio del Proyecto [21]. Adicionalmente, se han sintetizado unos enlaces a vídeos del catálogo en el Anexo A.4.

4.5 Análisis Global de Resultados Experimentales

La consolidación de los datos extraídos del catálogo de pruebas permite identificar patrones de comportamiento sistémico en el eje percepción-planificación. Los resultados se estructuran en función de la respuesta algorítmica frente a la topología del entorno, aislando las capacidades operativas de las limitaciones arquitectónicas críticas.

4.5.1 Navegación en Entornos de Baja y Media Complejidad

En escenarios urbanos con un índice de exigencia bajo o moderado ($M_{total} < 10.0$), el sistema exhibe una alta resiliencia topológica. En las pruebas ejecutadas sobre la red ortogonal simple (Town02), el algoritmo geométrico recalcula sistemáticamente las trayectorias frente a obstrucciones frontales y transversales, manteniendo una tasa de finalización de ruta del 100%.

El modelo multimodal demuestra una notable eficacia en la desagregación y agrupación semántica de los elementos del entorno. Se valida su capacidad para procesar múltiples objetos individuales contiguos (como alineaciones de conos de balizamiento o barreras modulares), integrándolos bajo una única entidad semántica de bloqueo. Este comportamiento optimiza la definición del *drivable path*, ya que el VLM infiere la restricción global de la vía sin requerir la detección unitaria de cada elemento bloqueante. Además, el rendimiento visual del sistema se mantiene estable ante los cambios de inclinación (*pitch*) en pendientes ascendentes y descendentes. Esto confirma que las variaciones naturales en la línea del horizonte, producidas por las subidas o bajadas del terreno, no reducen la precisión con la que el modelo detecta los obstáculos.

4.5.2 Anomalías Perceptuales y Falsos Positivos Sistémicos

La experimentación muestra vulnerabilidades recurrentes en el módulo de visión vinculadas a la resolución espacial y a la falta de contexto tridimensional. El proceso de segmentación y recorte de las imágenes, implementado para adecuar el campo de visión a la relación de aspecto del VLM, introduce distorsiones geométricas periféricas. En casos específicos, esta deformación de los píxeles en las esquinas del frame induce alucinaciones semánticas, provocando que el modelo reporte obstáculos inexistentes en la calzada.

En vías interurbanas y entornos de alta velocidad (como el mapa Town04), el sistema presenta problemas importantes al analizar las formas. El modelo da más peso a la forma geométrica de los objetos que a su posición real, lo que lo lleva a confundir elementos ubicados en los márgenes de la carretera (como barriles de impacto, muros de contención o rocas) con obstáculos directos que bloquean el paso.

Frente a estas alucinaciones perceptuales, el planificador (RouteManager) demuestra un comportamiento algorítmicamente resiliente. Al recibir la confirmación de bloqueo (falso positivo), el enrutador penaliza los pesos de los nodos afectados y genera trayectorias alternativas. Aunque estas

rutas resultan topológicamente subóptimas, el agente logra converger hacia las coordenadas de destino, garantizando la fluidez y la seguridad cinemática del vehículo frente a la degradación del estímulo visual.

4.5.3 Limitaciones en Geometrías Restrictivas y Casos de Colisión

Los escenarios de mayor dificultad ($M_{total} \geq 10.0$) muestran las limitaciones más críticas del sistema, provocando colisiones o la interrupción total de la navegación. Se ha detectado un fallo constante al procesar trayectorias curvas. En concreto, el modelo visual no logra interpretar correctamente los pasillos delimitados por balizas continuas (por ejemplo conos) que restringen al mismo tiempo el paso frontal y lateral en una misma imagen. Ante este tipo de geometrías, el sistema no calcula una nueva ruta y mantiene su trayectoria original, lo que termina en un choque directo contra el balizamiento.

En el ámbito algorítmico, la heurística actual de seguimiento temporal y espacial no es lo bastante precisa para gestionar zonas con muchas posibles direcciones o cruces complejos. El sistema es incapaz de discriminar de forma determinista cuándo una detección corresponde a un obstáculo previamente registrado (que debería ignorarse en ciclos de evaluación posteriores) y cuándo representa una nueva entidad bloqueante. Esta carencia fuerza intervenciones manuales (override) en escenarios de bloqueos irregulares a ambos lados de la vía.

Por último, en zonas con poco espacio para maniobrar, como las rotondas del mapa Town03, el gran tamaño de la infraestructura central (como los monumentos) provoca falsos positivos en el modelo visual. Este error hace que el sistema intente calcular una nueva ruta demasiado tarde. Como dentro de la rotonda no hay caminos alternativos viables para esquivar el supuesto obstáculo, el vehículo (BehaviorAgent) se queda sin espacio físico para reaccionar y acaba chocando inevitablemente contra los límites de la calzada.

4.6 Conclusiones

La evaluación experimental confirma que combinar el modelo multimodal Llama4 Scout con un planificador geométrico es una solución viable para que un vehículo autónomo comprenda el entorno. El sistema es muy fiable a la hora de detectar a tiempo obstáculos aislados en la carretera. A partir de esta información visual, el planificador es capaz de calcular nuevas rutas a gran escala, descartando los caminos bloqueados y buscando alternativas dentro de la red de carreteras.

A pesar de este buen funcionamiento en la detección y en el cambio de rutas principales, los experimentos también han revelado varios problemas críticos en el módulo de visión. En primer lugar, se han registrado fallos en curvas delimitadas; el modelo no es capaz de interpretar ni esquivar pasillos estrechos formados por balizas continuas, lo que provoca choques directos en estos tramos curvos.

Además, se observa una clara falta de precisión en tres dimensiones. El sistema tiene dificultades para calcular la profundidad real y los límites exactos de la zona transitable. Al no contar con coordenadas precisas, el modelo confunde frecuentemente elementos propios de los márgenes de la vía (como barriles de impacto o rocas) con obstáculos reales ubicados en mitad de la calzada.

Por último, el sistema carece de un seguimiento continuo de los objetos en el tiempo. Al no registrar los elementos de forma constante, el programa no puede distinguir con seguridad si un obstáculo ya se había detectado antes o si es una amenaza nueva. Esta limitación técnica acaba descoordinando las maniobras del vehículo, sobre todo cuando ocurren varios imprevistos al mismo tiempo en un cruce.

En conclusión, los resultados muestran que el sistema está muy preparado para la navegación general a gran escala, ya que sabe reaccionar y buscar rutas alternativas con eficacia ante los imprevistos. Sin

embargo, depender únicamente de un modelo visual (VLM) no ofrece la precisión necesaria para garantizar la seguridad en las maniobras cortas y precisas. Para que este sistema pueda usarse en el mundo real, es necesario cambiar a un diseño mixto que combine la capacidad de comprensión del modelo visual con sensores que midan las distancias reales (como el LiDAR o cámaras de profundidad). De este modo, se podrán definir los límites de los obstáculos de forma exacta y sin ambigüedades.

Capítulo 5

Conclusiones y líneas futuras

5.1 Conclusiones

5.1.1 Evaluación de Objetivos y Viabilidad del Sistema

El propósito fundamental de este Trabajo de Fin de Grado ha sido el diseño, desarrollo y validación de una arquitectura de navegación autónoma capaz de unificar el razonamiento semántico abstracto proporcionado por Modelos Multimodales de Visión y Lenguaje con algoritmos deterministas de planificación de trayectorias en entornos simulados de alta fidelidad como CARLA. Tras la finalización de la fase de experimentación y análisis técnico, se constata el cumplimiento satisfactorio de los objetivos generales y específicos planteados en la memoria inicial.

El sistema desarrollado ha demostrado viabilidad técnica al resolver con éxito escenarios complejos de bloqueo de calzada y desvíos imprevistos en mapas con diferentes configuraciones topológicas y densidades geométricas (Town02 y Town10HD). El cierre conceptual de la investigación valida la hipótesis de partida: es posible delegar la comprensión contextual y la detección de anomalías estructurales en la vía a un modelo fundacional multimodal de lenguaje de gran tamaño, traduciendo sus salidas cualitativas en comandos cuantitativos de modificación de grafos que el planificador local y global pueden ejecutar de manera segura y eficiente.

5.1.2 Contribuciones Tecnológicas a la Arquitectura de Navegación

La principal aportación técnica reside en la creación de una capa de abstracción intermedia entre la percepción cognitiva y el control físico, materializada en tres componentes de software originales. El primero consiste en una arquitectura orquestadora que implementa un bucle de control síncrono. Este módulo captura el flujo de vídeo del vehículo ego, gestiona la comunicación externa mediante peticiones estructuradas hacia la API de Groq y decodifica las respuestas en eventos semánticos en tiempo de ejecución.

El segundo componente es un gestor de penalizaciones topológicas que sustituye la replanificación binaria tradicional por una alteración dinámica de la matriz de costes en el grafo vial. Al aplicar estas penalizaciones sobre los tramos bloqueados, se obliga al algoritmo a calcular rutas alternativas sin afectar a la conectividad general de la red. El tercer componente es un traductor espacial, encargado de convertir lo que detecta el modelo de visión (VLM) en coordenadas exactas. Mediante el análisis de la posición y la geometría, este módulo se asegura de penalizar únicamente las zonas donde se encuentra

el obstáculo. En las pruebas experimentales, el sistema ha logrado completar con éxito la gran mayoría de las trayectorias analizadas, demostrando una sólida sincronización entre la detección anticipada y las maniobras del vehículo. No obstante, los resultados evidencian que la tasa de éxito no es absoluta, registrándose colisiones en escenarios específicos de alta exigencia. En concreto, el sistema presenta limitaciones operativas al enfrentarse a corredores balizados continuos o al verse forzado a gestionar intersecciones con múltiples salidas bloqueadas de forma simultánea.

5.1.3 Análisis de Robustez y Limitaciones Operativas

A pesar del buen rendimiento del sistema, es necesario señalar sus limitaciones actuales para evaluar su viabilidad en entornos reales.

El factor limitante más crítico, evidenciado durante la experimentación, es la vulnerabilidad del sistema ante escenarios de alta complejidad geométrica. Concretamente, la arquitectura de percepción actual presenta dificultades severas al procesar trayectorias canalizadas mediante conos (corredores balizados) o al gestionar intersecciones con múltiples vías bloqueadas de forma simultánea. En estas situaciones, el modelo no logra interpretar correctamente las restricciones espaciales continuas, lo que provoca una falta de reacción o una replanificación tardía que termina en colisión.

Un segundo factor a tener en cuenta son los errores ocasionales y los falsos positivos del modelo de visión. La evaluación revela que elementos del entorno (como rocas, barriles de autopista o monumentos) y ciertas perspectivas complejas en las cámaras provocan detecciones erróneas. Aunque estas falsas alarmas generan cambios de ruta preventivos que no comprometen la seguridad física del vehículo, sí que penalizan de forma directa la eficiencia energética y la fluidez de la trayectoria al trazar desvíos innecesarios.

Finalmente, a nivel de software, la estabilidad del sistema depende en gran medida de que el modelo de Inteligencia Artificial Generativa devuelva la información exactamente en el formato programado. Pequeñas desviaciones en la sintaxis de las respuestas obligan a incluir mecanismos de control de errores en el código principal, lo que reduce la flexibilidad y la autonomía del sistema frente a situaciones completamente imprevistas.

5.2 Líneas futuras

La arquitectura desarrollada establece los cimientos de un paradigma prometedor en la robótica autónoma, abriendo múltiples vertientes de expansión técnica alineadas con el estado del arte contemporáneo en inteligencia artificial y control vehicular.

5.2.1 Optimización del Módulo de Percepción y Edge AI

Para mitigar la limitación del procesamiento en la nube y garantizar un determinismo temporal compatible con los estándares de seguridad de la industria automotriz, la línea futura más inmediata consiste en la migración de los modelos multimodales a un despliegue local embebido:

- **Cuantización y Optimización de Pesos:** aplicar formatos reducidos (INT4, INT8 o FP16) mediante herramientas como TensorRT-LLM o vLLM [28]. El objetivo es desplegar modelos de lenguaje visual compactos (3B-8B parámetros) directamente en hardware embebido.
- **Fine-Tuning:** utilizar el modelo principal de este TFG como 'profesor' para destilar sus capacidades de razonamiento espacial en un modelo de visión más ligero y especializado [29]. Para ello, se

propone generar un conjunto de datos sintético en CARLA, con auto-etiquetado topológico, que permita entrenar un codificador visual optimizado específicamente para la conducción urbana.

5.2.2 Refinamiento de la Capa de Planificación, Grafos Dinámicos y Control Predictivo

Desde la perspectiva de la teoría de control y la optimización de trayectorias, la arquitectura actual basada en grafos estáticos puede evolucionar hacia formulaciones dinámicas más sofisticadas. En primer lugar, se propone sustituir la recalculación completa de la ruta (`GlobalRoutePlanner`) por algoritmos de búsqueda incremental en tiempo real, como D* [30] Lite o Any-Angle Path Planning. Esto reduciría drásticamente el coste computacional al actualizar únicamente las zonas del grafo afectadas por un evento semántico, evitando reevaluar el trayecto desde cero.

En segundo lugar, el guiado del vehículo puede mejorarse sustituyendo el control PID por un Control Predictivo por Modelo [31]. Al tener en cuenta la física del vehículo, este controlador consigue suavizar las maniobras bruscas y adaptar la velocidad según el riesgo que detecte el modelo visual. Por último, se propone añadir la dimensión del tiempo al mapa de navegación (X, Y, Tiempo). De este modo, además de esquivar obstáculos fijos, el sistema podría anticipar hacia dónde se van a mover los peatones u otros coches, ajustando la ruta con antelación.

5.2.3 Ampliación del Marco de Validación y Escenarios Cooperativos

Finalmente, para asegurar la capacidad de generalización de la arquitectura, resulta crucial evaluar el sistema en entornos de simulación más exigentes. En primer lugar, se propone validar el codificador visual del modelo (VLM) frente a condiciones climáticas adversas en CARLA, tales como lluvia densa, niebla, destellos solares o conducción nocturna. Este análisis permitirá medir la robustez de su razonamiento semántico ante caídas severas en la calidad de la imagen.

En segundo lugar, se plantea la integración de arquitecturas cooperativas, donde múltiples vehículos actúen como nodos sensores interconectados [32]. De este modo, si un vehículo detecta un obstáculo y penaliza su mapa topológico local, puede transmitir esta alteración al resto de la flota o a la nube mediante protocolos de comunicación automotriz. Esto habilitaría una replanificación de trayectorias predictiva y a nivel global, incluso antes de que los demás vehículos tengan contacto visual directo con la obstrucción.

Bibliografía

- [1] E. Yurtsever, J. Lambert, A. Carballo y K. Takeda, “A Survey of Autonomous Driving: Common Practices and Emerging Technologies”, *IEEE Access*, vol. 8, págs. 58 443-58 469, 2020.
- [2] T. G. R. Reid, S. E. Houts, R. Cammarata, G. Chowdhary y J. C. Becker, “Localization Requirements for Autonomous Vehicles”, *SAE International Journal of Connected and Automated Vehicles*, vol. 2, n° 3, págs. 173-190, 2019.
- [3] C. Cadena et al., “Past, Present, and Future of Simultaneous Localization and Mapping: Toward the Robust-Perception Age”, *IEEE Transactions on Robotics*, vol. 32, n° 6, págs. 1309-1332, dic. de 2016.
- [4] H. G. Seif y X. Hu, “Autonomous Driving in the iCity—HD Maps as a Key Challenge of the Automotive Industry”, *Engineering*, vol. 2, n° 2, págs. 159-162, 2016.
- [5] M. Sato, R. Kobayashi, K. Nomoto, Y. Tanaka, G. Tsuruoka y T. Mori, “WIP: Evaluation of Threats and Impacts of HD Map Tampering Attacks in Autonomous Driving”, en *3rd USENIX Symposium on Vehicle Security and Privacy (VehicleSec 25)*, <https://www.usenix.org/conference/vehiclesec25/presentation/sato>, 2025.
- [6] Y. Tang et al., “CausalVAD: De-confounding End-to-End Autonomous Driving via Causal Intervention”, en *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, <https://arxiv.org/pdf/2603.18561>, 2026.
- [7] L. Chen et al., “Driving with LLMs: Fusing Object-Level Vector Modality for Explainable Autonomous Driving”, en *2024 IEEE International Conference on Robotics and Automation (ICRA)*, 2024.
- [8] Y. Ma et al., “LaMPilot: An Open Benchmark Dataset for Autonomous Driving with Language Model Programs”, en *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [9] V. Lialin, V. Deshpande y A. Rumshisky, “Scaling Down to Scale Up: A Guide to Parameter-Efficient Fine-Tuning”, *arXiv preprint arXiv:2303.15647*, 2023.
- [10] E. J. Hu et al., “LoRA: Low-Rank Adaptation of Large Language Models”, en *International Conference on Learning Representations (ICLR)*, <https://arxiv.org/abs/2106.09685>, 2022.
- [11] Y. Wang et al., “Drive as Veteran: Fine-tuning of an Onboard Large Language Model for Highway Autonomous Driving”, en *2024 IEEE Intelligent Vehicles Symposium (IV)*, <https://ieeexplore.ieee.org/document/10588851>, 2024.
- [12] X. Tian et al., “DriveVLM: The Convergence of Autonomous Driving and Large Vision-Language Models”, *arXiv preprint arXiv:2402.12289*, 2024.

- [13] B. Yang, B. Yao, T. Fu y H. Du, “ThinkDrive: Adaptive Dual-Process Reasoning for Autonomous Driving”, *Mathematics*, 2025.
- [14] A. L. Ballardini y M. Á. Sotelo, “Human-like Semantic Navigation for Autonomous Driving using Knowledge Representation and Large Language Models”, *arXiv preprint arXiv:2505.16498*, 2025.
- [15] P. E. Hart, N. J. Nilsson y B. Raphael, “A Formal Basis for the Heuristic Determination of Minimum Cost Paths”, *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, n° 2, págs. 100-107, jul. de 1968.
- [16] *Página oficial del simulador CARLA (Open-source simulator for autonomous driving research)*, <https://carla.org/> [Último acceso 8/julio/2026].
- [17] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez y V. Koltun, “CARLA: An Open Urban Driving Simulator”, en *Proceedings of the 1st Annual Conference on Robot Learning (CoRL)*, <https://arxiv.org/abs/1711.03938>, 2017, págs. 1-16.
- [18] *Página oficial del motor gráfico Unreal Engine*, <https://www.unrealengine.com/> [Último acceso 8/julio/2026].
- [19] *Referencia de la API de Python del Simulador CARLA*, https://carla.readthedocs.io/en/latest/python_api/ [Último acceso 8/julio/2026].
- [20] *Catálogo de Props del Simulador CARLA*, https://carla.readthedocs.io/en/latest/catalogue_props/ [Último acceso 8/julio/2026].
- [21] Rubén y Gabriel, *Repositorio de código del proyecto: Estudio e Implementación de un Planificador de Trayectorias en el Simulador CARLA*, https://github.com/invett/TFG.BAL.002.CARLA_RUBEN_GABRIEL.git [Último acceso 8/julio/2026].
- [22] AI@Meta, *Model Card for Llama 4 Scout 17B 16E Instruct*, <https://huggingface.co/meta-llama/llama-4-scout-17b-16e-instruct> [Último acceso 8/julio/2026], 2026.
- [23] A. Dosovitskiy et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale”, en *International Conference on Learning Representations (ICLR)*, <https://arxiv.org/abs/2010.11929>, 2021.
- [24] H. Liu, C. Li, Q. Wu e Y. J. Lee, “Visual Instruction Tuning”, en *Advances in Neural Information Processing Systems (NeurIPS)*, <https://arxiv.org/abs/2304.08485>, 2024.
- [25] A. Q. Jiang et al., “Mixtral of Experts”, *arXiv preprint arXiv:2401.04088*, 2024.
- [26] J. White et al., “A Prompt Pattern Catalog to Enhance Prompt Engineering with ChatGPT”, *arXiv preprint arXiv:2302.11382*, 2023.
- [27] J. Wei et al., “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models”, en *Advances in Neural Information Processing Systems (NeurIPS)*, <https://arxiv.org/abs/2201.11903>, 2022.
- [28] W. Kwon et al., “Efficient Memory Management for Large Language Model Serving with PagedAttention”, en *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023.
- [29] G. Hinton, O. Vinyals y J. Dean, “Distilling the Knowledge in a Neural Network”, *arXiv preprint arXiv:1503.02531*, 2015.
- [30] S. Koenig y M. Likhachev, “D* Lite”, en *Eighteenth National Conference on Artificial Intelligence (AAAI)*, 2002, págs. 476-483.

-
- [31] P. Falcone, F. Borrelli, J. Asgari, H. E. Tseng y D. Hrovat, “Predictive Active Steering Control for Autonomous Vehicle Systems”, *IEEE Transactions on Control Systems Technology*, vol. 15, n° 3, págs. 566-580, mayo de 2007.
- [32] S. Chen et al., “Vehicle-to-Everything (V2X) Services Supported by LTE-Based Systems and 5G”, *IEEE Communications Standards Magazine*, vol. 1, n° 2, págs. 70-76, 2017.

Apéndice A

Manual de usuario

A.1 Introducción

El presente manual de usuario tiene como objetivo principal guiar al lector en el proceso de instalación, configuración y ejecución del sistema desarrollado en este Trabajo de Fin de Grado.

Para garantizar la reproducibilidad y facilitar la curva de aprendizaje, este capítulo detalla paso a paso las dependencias necesarias, el proceso de despliegue y proporciona ejemplos prácticos de uso, así como material audiovisual de apoyo.

A.2 Requisitos previos e Instalación

Para garantizar la correcta ejecución del entorno de simulación autónoma y evitar conflictos de compatibilidad, es indispensable preparar el entorno de trabajo siguiendo una configuración específica. A continuación, se detallan los requisitos de software, la gestión de dependencias y la configuración de las credenciales de inteligencia artificial.

A.2.1 Software base necesario

El núcleo de la simulación se apoya en **CARLA Simulator**. Debido a la arquitectura de su API y a la compatibilidad de las librerías empleadas, el sistema ha sido desarrollado y validado para funcionar sobre las versiones **0.9.14** y **0.9.15** de CARLA. Asimismo, es necesario disponer de un intérprete de **Python** (versión **3.7** o superior).

Nota importante de configuración: El sistema está diseñado para buscar los módulos nativos de CARLA directamente en el equipo del usuario. Por ello, antes de realizar cualquier prueba, es obligatorio actualizar la ruta absoluta de la instalación de CARLA. Esto debe realizarse en las primeras líneas de los scripts `vlm_simulation_runner.py` y `planning/Navigation.py`, modificando la siguiente instrucción para que apunte al directorio correcto:

```
import sys
# El usuario debe ajustar esta ruta a su directorio de CARLA
sys.path.append('C:/Ruta/A/CARLA_0.9.15/WindowsNoEditor/PythonAPI/carla')
```

A.2.2 Instalación de dependencias

El proyecto hace uso de múltiples librerías externas de Python destinadas a la visión artificial (`opencv-python`), la lectura de configuraciones (`PyYAML`), los cálculos topológicos (`networkx`) y la comunicación HTTP (`requests`).

Para asegurar un despliegue limpio y unificado, todas las dependencias necesarias han sido encapsuladas en el fichero `requirements.txt`. Se recomienda utilizar un entorno virtual para no interferir con otras instalaciones del sistema. Desde la consola, en el directorio raíz del proyecto, se debe ejecutar:

```
$ pip install -r requirements.txt
```

A.2.3 Configuración de la clave de la API de Groq

El módulo de percepción visual (VLM) requiere acceso en tiempo real a los modelos de inteligencia artificial alojados en la infraestructura de Groq. Para autorizar la transmisión de las imágenes y la recepción de las inferencias semánticas, es estrictamente necesario inyectar una clave de acceso válida (API Key).

El usuario debe acceder a la plataforma para desarrolladores de Groq (<https://console.groq.com/keys>) y generar una nueva credencial. Una vez obtenida, esta debe integrarse en la cabecera del orquestador principal (`vlm_simulation_runner.py`), reemplazando el valor de la variable global `GROQ_API_KEY` por la clave personal:

```
# — GROQ CONFIGURATION —  
GROQ_API_KEY = "TU_CLAVE_API_AQUI"  
GROQ_URL = "https://api.groq.com/openai/v1/chat/completions"
```

Sin esta configuración, el hilo de procesamiento de inteligencia artificial (`vlm_worker`) lanzará excepciones de conexión y el vehículo no será capaz de percibir los obstáculos del entorno.

A.3 Ejecución de la Simulación

Una vez completada la configuración del entorno y las credenciales, el sistema está preparado para ejecutar las misiones de conducción autónoma. El proceso de inicialización consta de dos fases: el arranque del servidor de renderizado y el lanzamiento del orquestador algorítmico.

A.3.1 Arranque del sistema

En primer lugar, es obligatorio iniciar el servidor del simulador CARLA. Dependiendo del sistema operativo, se debe ejecutar el binario correspondiente (por ejemplo, `CarlaUE4.exe` en Windows o `./CarlaUE4.sh` en Linux). Se recomienda esperar unos segundos hasta que el entorno 3D esté completamente cargado.

A continuación, desde una terminal en el directorio raíz del proyecto y con el entorno virtual activado, se lanza el script principal que orquesta la simulación:

```
$ python vlm_simulation_runner.py
```

A.3.2 Menú interactivo de configuración

Al ejecutar el orquestador, el sistema leerá los archivos YAML de configuración (`obstacles_eval.yaml` e `inicio_fin.yaml`) y desplegará un menú interactivo en la consola. Este menú facilita la selección de los casos de prueba sin necesidad de modificar el código fuente.

El usuario se encontrará con dos solicitudes secuenciales:

1. **Selección de Escenario:** se presentará una lista numerada con los identificadores de los escenarios disponibles y una breve descripción. El usuario debe introducir el número correspondiente a la ruta que desea evaluar (o la tecla `Q` para cancelar la ejecución).
2. **Instanciación Física de Obstáculos (*Spawning*):** el sistema preguntará si se desea materializar los obstáculos en el mapa 3D (Y/N). Seleccionar "Y" inyectará físicamente elementos como barreras, conos o vehículos policiales en las coordenadas indicadas por el archivo YAML.

Una vez confirmados ambos pasos, el script se conectará al cliente de CARLA, instanciará el vehículo (*ego-vehicle*), inicializará el planificador de rutas y comenzará la simulación.

A.3.3 Interacción en tiempo real (Override Manual)

Durante la ejecución, el sistema abrirá una ventana gráfica (gestionada mediante OpenCV) que mostrará la cámara frontal del vehículo y un *Head-Up Display* (Figura 3.11) con la telemetría, el estado del modelo multimodal y el registro de razonamiento de la IA.

Se ha habilitado un mecanismo de interrupción manual (*Manual Override*) para inyectar eventos de percepción forzados en el sistema. Esto resulta especialmente útil para evaluar la capacidad de replanificación del RouteManager frente a fallos del VLM o situaciones de estrés geométrico. Estando la ventana de visualización activa, el usuario puede utilizar los siguientes comandos de teclado:

- **Tecla I:** inyecta un evento manual de bloqueo **Frontal**. El sistema interpretará que hay un obstáculo inminente en la trayectoria recta.
- **Tecla J:** inyecta un evento manual de bloqueo en el flanco **Izquierdo**. Útil en intersecciones.
- **Tecla L:** inyecta un evento manual de bloqueo en el flanco **Derecho**.
- **Teclas Q o ESC:** detienen la simulación inmediatamente, destruyen los actores instanciados de forma segura y cierran la ventana gráfica.

Al accionar las teclas de *override*, el HUD actualizará el estado de la fuente a "OVERRIDE" y el planificador intentará buscar automáticamente una ruta alternativa basándose en la nueva restricción topológica impuesta.

A.4 Catálogo de Pruebas y Demostraciones Visuales

Para garantizar la transparencia y permitir la validación empírica de los resultados obtenidos en este Trabajo de Fin de Grado, se ha elaborado un documento exhaustivo denominado `catalogo.md`, ubicado en la raíz del repositorio.

Dicho catálogo recoge la evaluación técnica de todos los escenarios de prueba (identificados mediante el prefijo TC-), abarcando desde entornos residenciales simples (Town02) hasta simulaciones de alta densidad urbana (Town10HD). Cada entrada del catálogo detalla la configuración del bloqueo, las métricas de percepción del modelo multimodal (falsos positivos/negativos), el rendimiento del planificador espacial y un análisis pormenorizado del comportamiento dinámico del vehículo.

Con el fin de ilustrar el funcionamiento del sistema en tiempo real y facilitar la evaluación por parte del tribunal, las ejecuciones han sido grabadas. A continuación, se presenta una selección representativa de los casos de prueba más relevantes. Utilizando el formato interactivo de esta memoria, puede acceder directamente a las demostraciones en vídeo haciendo clic en los siguientes enlaces:

Entornos Residenciales (Town02 y Town03)

[TC-TWN02-004: Variante topológica con múltiples obstáculos sucesivos](#) [TC-TWN03-003: Intervención multisectorial de alta complejidad \(bomberos, obras y policía\)](#)

Vías de Alta Capacidad y Tácticas Avanzadas (Town04)

[TC-TWN04-002: Identificación de balizamiento mimetizado frente a ruido visual de fondo](#)
[TC-TWN04-009: Detección de trampa topológica profunda y ejecución de maniobra de retorno](#)

Entorno Urbano de Alta Resolución (Town10HD)

[TC-TWN10HD-004: Triple bloqueo con detección crítica de obstáculo escurado en curva](#)
[TC-TWN10HD-008: Operación policial bajo condiciones visuales adversas \(sombras y baja iluminación\)](#)

Nota: El listado completo con todos los vídeos, descripciones detalladas de los escenarios y métricas de evaluación se encuentra disponible para su consulta interactiva en el archivo `catalogo.md` del repositorio oficial [21].

Apéndice B

Herramientas y tecnologías utilizadas

En este anexo se detallan las herramientas de hardware y software, los lenguajes de programación, las bibliotecas y los modelos de Inteligencia Artificial que han conformado la arquitectura tecnológica de este proyecto.

B.0.1 Hardware

- **Tarjeta Gráfica (GPU) – NVIDIA GeForce RTX 2060:** el simulador utilizado requiere de una alta capacidad de cómputo gráfico y procesamiento en tiempo real. La ejecución del entorno tridimensional, el renderizado de los sensores virtuales (cámaras RGB) y la gestión del cliente de simulación se han llevado a cabo apoyándose en una GPU NVIDIA RTX 2060. Esta tarjeta ha permitido mantener una tasa de fotogramas por segundo (FPS) estable y procesar la captura de imágenes simultáneas que posteriormente alimentan a la red neuronal.

B.0.2 Entorno y Sistemas Operativos

- **Entorno Multiplataforma (Windows y Linux):** el desarrollo, las pruebas y la ejecución de la arquitectura propuesta se han realizado de forma cruzada en los sistemas operativos Windows y Linux. Esto ha permitido garantizar la portabilidad del código, la compatibilidad de las rutas del simulador y la validación del sistema en diferentes entornos de ejecución.
- **CARLA Simulator (v0.9.14 / v0.9.15):** Plataforma de código abierto empleada como entorno principal de simulación para la conducción autónoma. CARLA ha proporcionado el motor físico, los mapas urbanos (*Town02*, *Town03*, *Town04*, *Town10HD*), el ecosistema de sensores y la API necesaria para controlar el ego-vehículo.

B.0.3 Lenguaje de Programación

- **Python 3.x:** lenguaje principal utilizado para orquestar toda la lógica del proyecto. Se ha utilizado para integrar la API de CARLA, gestionar la lógica de navegación, procesar las imágenes y realizar las peticiones al modelo fundacional.

B.0.4 Bibliotecas y Módulos de Python

El desarrollo de los distintos módulos (*RouteManager*, *ReplanningManager*, *PenaltyManager*, etc.) se ha fundamentado en las siguientes librerías de terceros y estándares:

B.0.4.1 Visión Artificial y Procesamiento de Datos

- **OpenCV (`cv2`):** fundamental para el procesamiento de los fotogramas capturados por la cámara RGB del vehículo. Se ha empleado para redimensionar las imágenes, generar recortes (*crops*) de la vista izquierda, central y derecha, codificarlas en formato JPEG y dibujar el *Head-Up Display* (HUD) de telemetría en tiempo real sobre la ventana de visualización.
- **NumPy (`numpy`):** utilizado exhaustivamente para el cálculo matricial y el álgebra lineal. En el proyecto se emplea para calcular distancias euclidianas entre *waypoints* (vectores geométricos), conversiones de los *buffers* de la cámara de CARLA a matrices de imagen, y operaciones con los vectores de dirección del coche.

B.0.4.2 Planificación de Rutas y Grafos

- **NetworkX (`networkx`):** librería utilizada como motor matemático para la representación de la topología vial. El planificador global convierte el mapa de CARLA en un grafo dirigido (`nx.DiGraph`) donde los nodos son coordenadas y las aristas (*edges*) son tramos de carretera. Se emplea su implementación del algoritmo A^* (`nx.astar_path`) para calcular la ruta más corta y gestionar los recálculos dinámicos al aplicar penalizaciones en los pesos de las aristas (*RoutePenaltyManager*).

B.0.4.3 Manejo de Archivos y Concurrencia

- **PyYAML (`yaml`):** empleado para la lectura y parseo de los archivos de configuración dinámicos (como `obstacles.yaml` y `inicio_fin.yaml`), separando así la definición de los casos de prueba de la lógica de código.
- **Threading (`threading`) y Queue (`queue`):** librerías estándar de Python utilizadas para implementar el procesamiento asíncrono. Permiten separar el hilo de ejecución principal (el bucle del simulador que corre a un *tick* fijo) del hilo encargado de la inferencia de la IA (`vlm_worker`), evitando cuellos de botella y latencias (bloqueos) durante las peticiones de red.
- **JSON y Base64 (`json`, `base64`):** usados para la estructuración, codificación y decodificación de la información (las imágenes procesadas) al formato requerido por la API del modelo de lenguaje.

B.0.5 Inteligencia Artificial y Modelos Multimodales (VLM)

- **Groq API:** plataforma de inferencia en la nube empleada para acelerar los tiempos de respuesta del modelo (reduciendo drásticamente la latencia de inferencia en comparación con la ejecución puramente local en CPU/GPU).
- **Modelo VLM (Visual Language Model):** para la tarea crítica de percepción y razonamiento espacial (extraer el contexto, detectar obstáculos y determinar qué carriles están libres), se ha implementado el modelo multimodal especificado en la integración

(meta-llama/llama-4-scout-17b-16e-instruct). Este modelo recibe un *prompt* estructurado junto con tres perspectivas codificadas de la vía y devuelve un JSON que el sistema procesa como un objeto de tipo `PerceptionEvent`.

B.0.6 Herramientas de Documentación y Control de Pruebas

- **Markdown:** el lenguaje de marcado Markdown se ha utilizado para redactar el catálogo de casos de prueba (`catalogo.md`). Gracias a él se ha documentado exhaustivamente cada iteración de la simulación, incorporando métricas de evaluación (% de ruta completado, falsos positivos/negativos, latencia), tablas de dificultad y recursos visuales (vídeos) representativas del comportamiento del vehículo frente a distintos bloqueos topológicos (estructuras, conos, retenes policiales, etc.).

B.0.7 Control de Versiones y Gestión del Proyecto

Para garantizar la integridad del código fuente, el registro histórico de las modificaciones y la correcta planificación de las tareas, se han integrado las siguientes herramientas:

- **Git y GitHub:** se ha empleado Git como sistema de control de versiones distribuido, alojando el repositorio central en la plataforma GitHub. Esto ha permitido mantener un historial estructurado de los avances del proyecto (mediante *commits*), para la implementación de nuevas características y asegurar la trazabilidad completa del código.
- **GitHub Projects (Gestión de *Issues*):** la planificación y el seguimiento del ciclo de vida del desarrollo se han orquestado utilizando GitHub Projects. Mediante un sistema de tableros y la creación de *issues*, se ha llevado a cabo la organización de las tareas, la priorización de los módulos a programar y la documentación de los errores (*bugs*) detectados durante las simulaciones. Cada *issue* resuelta se ha vinculado a los cambios específicos en el código, garantizando un flujo de desarrollo ordenado y metodológico.



Universidad
de Alcalá

2026